UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS





INFORME DEL PROYECTO DE SISTEMAS OPERATIVOS

Algoritmos de reemplazo de páginas

CURSO: Sistemas Operativos

DOCENTE: Ing. Emilio Palomino Olivera

INTEGRANTES:

- Angel Ismael Alvarez Catunta 221945
- Haider Alex Carpio Hermoza
- Jhamil Jhonatan Huahualuque Vargas
- Marcus Geuseppe Meza Zamalloa
- Elvis Raúl Suca Hancco

CUSCO - PERÚ

Semestre 2026 - II

INTRODUCCIÓN

Descripción y evolución del tema

La memoria principal ha sido, desde los primeros computadores, un recurso escaso frente a las necesidades de los programas. La respuesta de los sistemas operativos fue la **memoria virtual**: una abstracción que permite ejecutar procesos cuyo espacio de direcciones lógico supera la memoria física disponible, manteniendo en RAM solo las porciones que se están utilizando y el resto en el almacenamiento secundario. Su primera implementación fue el sistema de almacenamiento de un solo nivel del computador Atlas, en la Universidad de Mánchester (Kilburn et al., 1962).

La técnica que hizo práctica esta idea es la **paginación por demanda**: el espacio lógico se divide en páginas de tamaño fijo y la memoria física en marcos del mismo tamaño, y cada página se carga únicamente cuando el proceso la referencia (Silberschatz et al., 2018). Cuando una página referenciada no está en memoria ocurre un **fallo de página**, y si todos los marcos están ocupados el sistema operativo debe elegir una página **víctima** para liberar un marco. La regla que hace esa elección es el **algoritmo de reemplazo de páginas**.

El estudio de estos algoritmos tiene una larga trayectoria. Belady (1966) caracterizó el algoritmo óptimo (MIN) y comparó estrategias realizables; Belady et al. (1969) descubrieron que FIFO puede producir más fallos al recibir más memoria; Denning (1968) formalizó el modelo del conjunto de trabajo y explicó la hiperpaginación; Corbató (1969) introdujo el algoritmo del reloj en Multics y Carr y Hennessy (1981) lo combinaron con el conjunto de trabajo en WSClock. En el hardware moderno, aunque las unidades NVMe redujeron la latencia del almacenamiento a decenas de microsegundos, la distancia con la RAM (cerca de 100 ns) sigue siendo de dos a tres órdenes de magnitud, por lo que los núcleos actuales continúan usando aproximaciones eficientes de LRU (Tanenbaum & Bos, 2015).

Problema

Cada fallo de página interrumpe al proceso, provoca una operación de entrada/salida y, si la página expulsada fue modificada, una escritura adicional a disco. El impacto se mide con el tiempo efectivo de acceso: con un acceso a memoria de 200 ns y un servicio de fallo de 8 ms, basta un

fallo cada 1 000 accesos para que el tiempo efectivo suba a unos 8.2 µs, es decir, la memoria se vuelve unas 40 veces más lenta (Silberschatz et al., 2018).

Cuando los procesos disponen de menos marcos que los que exige su localidad de referencia, el sistema entra en **hiperpaginación** (*thrashing*): dedica más tiempo a mover páginas entre disco y memoria que a ejecutar instrucciones útiles (Denning, 1968). Un algoritmo de reemplazo inadecuado agrava este fenómeno, mientras que uno bien elegido lo mitiga sin necesidad de más hardware.

Propuesta de solución

El presente proyecto propone un **análisis empírico y comparativo de 12 algoritmos de reemplazo de páginas** bajo condiciones idénticas de contención de memoria: **3 marcos físicos** y una **cadena fija de 30 referencias** con un patrón definido de lecturas y escrituras. Para ello se desarrolló un simulador modular en lenguaje C para Linux que registra, paso a paso, el contenido de los marcos, los fallos, los aciertos, las variables internas de cada algoritmo y las escrituras a disco.

Los resultados fueron validados con una implementación de referencia independiente y con 67 pruebas automáticas (todas superadas), y se complementan con un análisis de sensibilidad (1 a 8 marcos) y con la verificación experimental de la anomalía de Belády.

Finalmente, el mismo código C se compiló a **WebAssembly** para ofrecer un **simulador web interactivo**, que permite reproducir paso a paso cada algoritmo con cualquier cadena y número de marcos.

ÍNDICE

INTRODUCCIÓN **1**
Descripción y evolución del tema 1
Problema 1
Propuesta de solución 2
CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA Y METODOLOGÍA **4**
1.1 Planteamiento del problema 4
1.1.1 Descripción del problema 4
1.1.2 Identificación del problema 4
1.2 Justificación 5
1.3 Limitaciones 5
1.4 Delimitaciones 6
1.5 Metodología 6
1.5.1 Cronograma 7
CAPÍTULO II: MARCO TEÓRICO Y MODELADO DE LOS ALGORITMOS **9**
2.1 Antecedentes internacionales 9
2.2 Antecedentes nacionales 10
2.3 Temas relacionados al proyecto con modelado de cada algoritmo 10
2.3.1 Fundamentos de la memoria virtual y condiciones del experimento 10
2.3.2 Algoritmo 1: FIFO (First-In, First-Out) 12
2.3.3 Algoritmo 2: Óptimo (OPT / MIN de Belády) 14
2.3.4 Algoritmo 3: LRU (Least Recently Used) 18
2.3.5 Algoritmo 4: Reloj o Segunda Oportunidad (Clock) 20
2.3.6 Algoritmo 5: NRU (Not Recently Used) 23
2.3.7 Algoritmo 6: LFU (Least Frequently Used) 26
2.3.8 Algoritmo 7: MFU (Most Frequently Used) 28
2.3.9 Algoritmo 8: Reloj Mejorado (Enhanced Second Chance) 31
2.3.10 Algoritmo 9: Envejecimiento (Aging) 34
2.3.11 Algoritmo 10: Conjunto de Trabajo (Working Set) 37
2.3.12 Algoritmo 11: WSClock (Working Set Clock) 40
2.3.13 Algoritmo 12: Reemplazo Aleatorio (Random) 43
2.4 Evaluación comparativa de resultados 45
2.5 Validación del simulador 49
2.6 Simulador web interactivo 50
CONCLUSIONES **53**
RECOMENDACIONES 53
BIBLIOGRAFÍA 54
ANEXOS **56**
Anexo A. Código fuente común del simulador 56
Anexo B. Compilación y ejecución en Linux 68

CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA Y METODOLOGÍA

1.1 Planteamiento del problema

1.1.1 Descripción del problema

El cuello de botella de la memoria virtual es el fallo de página. Mientras la CPU accede a la RAM en nanosegundos, el almacenamiento secundario responde en micro o milisegundos; por ello, la eficiencia de todo el sistema depende de cuántas veces se recurre al disco. El algoritmo de reemplazo decide qué página abandona la memoria y, en consecuencia, cuántos fallos futuros se producirán.

El problema tiene un componente de **compromiso**: el algoritmo óptimo (OPT) minimiza los fallos, pero requiere conocer el futuro y no puede implementarse; LRU se acerca a él, pero su implementación exacta exige actualizar una marca de tiempo en cada acceso a memoria, lo que resulta costoso en hardware. Entre ambos extremos existe una familia de aproximaciones (Reloj, NRU, Aging, WSClock, entre otras) que equilibran la **complejidad algorítmica** con la **tasa de aciertos** (*hit ratio*) y con el costo de escritura de las páginas modificadas (Tanenbaum & Bos, 2015).

Elegir entre estas alternativas sin datos comparables conduce a decisiones basadas en intuición. Se requiere, por tanto, un estudio que evalúe los algoritmos bajo las mismas condiciones y con métricas reproducibles.

1.1.2 Identificación del problema

Los factores críticos que originan el problema son:

- **Disparidad de velocidad memoria–disco:** cada fallo cuesta miles de veces más que un acierto.
- **Memoria física limitada:** la cantidad de marcos es menor que el número de páginas del proceso.
- **Desconocimiento del futuro:** el algoritmo óptimo no es realizable; los algoritmos reales deben predecir el uso futuro a partir del pasado.
- **Costo de las aproximaciones:** LRU exacto es caro; las aproximaciones con bits R y M sacrifican precisión a cambio de eficiencia.

- **Páginas modificadas:** expulsar una página sucia agrega una escritura a disco.
- **Anomalía de Belády:** en algunos algoritmos, agregar marcos puede aumentar los fallos.
- **Hiperpaginación:** con pocos marcos, el sistema puede pasar más tiempo paginando que ejecutando.

1.2 Justificación

Justificación teórica. El proyecto integra los conceptos centrales de la gestión de memoria (paginación por demanda, localidad de referencia, conjunto de trabajo, algoritmos de pila y anomalía de Belády) y los contrasta con datos medidos, en lugar de limitarse a su descripción.

Justificación técnica. El simulador en C reproduce las estructuras que emplea un núcleo real (bits de referencia y modificación, punteros circulares, registros de desplazamiento) y produce trazas verificables. Su diseño modular permite agregar algoritmos y parámetros sin rehacer el código.

Justificación práctica. En la formación del Ingeniero Informático y de Sistemas, comprender cómo el núcleo administra la memoria es la base para optimizar servidores, bases de datos, sistemas embebidos y máquinas virtuales, donde las políticas de reemplazo (de páginas, de caché o de búferes) determinan el rendimiento.

1.3 Limitaciones

- La simulación se ejecuta en **espacio de usuario**; no modifica el núcleo de Linux ni mide tiempos reales de acceso a disco.
- Se usa una **única cadena estática de 30 referencias**; los resultados son válidos para esa cadena y no deben generalizarse a cualquier carga de trabajo.
- Se modela **un solo proceso con asignación fija** de marcos; no hay competencia entre procesos ni asignación global.
- No se modela **contención multinúcleo** ni concurrencia en tiempo real; el tiempo es virtual (1 referencia = 1 tic).
- El **patrón de escrituras** fue definido por el equipo para representar páginas de código y de datos; no proviene de la traza de un programa real.

- El resultado del algoritmo **Aleatorio** depende de la semilla (42); con otra semilla puede variar.
- La búsqueda de la anomalía de Belády en cadenas aleatorias es **limitada** (600 cadenas): no observarla no demuestra que un algoritmo sea inmune.

1.4 Delimitaciones

- **Conceptual:** reemplazo local de páginas en paginación por demanda; 12 algoritmos: FIFO, Óptimo, LRU, Reloj, NRU, LFU, MFU, Reloj Mejorado, Aging, Working Set, WSClock y Aleatorio.
- **Temporal:** Semestre académico 2026-II.
- **Hardware simulado:** 3 marcos de página físicos (con un análisis complementario de 1 a 8 marcos); el tamaño de página no interviene porque se simulan números de página.
- **Plataforma:** Linux Ubuntu 26.04 LTS sobre WSL 2, compilador gcc 15.2 con C estándar ISO C11 (`-std=c11 -Wall -Wextra -pedantic -O2`), GNU Make y Python 3 para validación y gráficas.
- **Espacial:** Escuela Profesional de Ingeniería Informática y de Sistemas, UNSAAC, Cusco.

1.5 Metodología

Enfoque: cuantitativo. **Tipo:** aplicado. **Diseño:** experimental por simulación, en el que se controlan todas las condiciones excepto la política de reemplazo.

Variable independiente: el algoritmo de reemplazo (12 niveles). En el análisis de sensibilidad, también el número de marcos (1 a 8). **Variables dependientes:** fallos de página, aciertos, tasa de fallos y escrituras a disco. **Variables controladas:** cadena de referencias, patrón de lecturas y escrituras, número de marcos, estado inicial y parámetros de cada algoritmo.

Procedimiento: (1) fijar los parámetros del experimento; (2) preparar el entorno Linux; (3) construir una estructura modular común; (4) implementar los 12 algoritmos; (5) validar; (6) generar trazas, métricas y gráficas; (7) analizar y redactar.

Recolección de datos: el propio simulador registra cada paso y exporta los resultados a archivos de texto y CSV, lo que evita la transcripción manual.

Validación: (a) valores de control calculados de forma independiente (FIFO = 25, OPT = 15, LRU = 22, Reloj = 24 fallos); (b) reproducción del ejemplo de Belády del material del curso; (c) comparación paso a paso contra una implementación de referencia escrita por separado en Python; (d) invariantes, casos de borde y 400 casos aleatorios. Resultado: 67 de 67 pruebas superadas, incluidas las del simulador web.

Control de versiones: todo el trabajo se registró en un repositorio Git con una etiqueta por cada fase (`paso-01`, `paso-02`, etc.).

1.5.1 Cronograma

La Tabla 1 detalla las fases, tareas, duración (en días hábiles), recursos asignados y predecesoras, en el formato de una hoja de tareas de MS Project. La Figura 1 muestra el diagrama de Gantt resultante.

Tabla 1

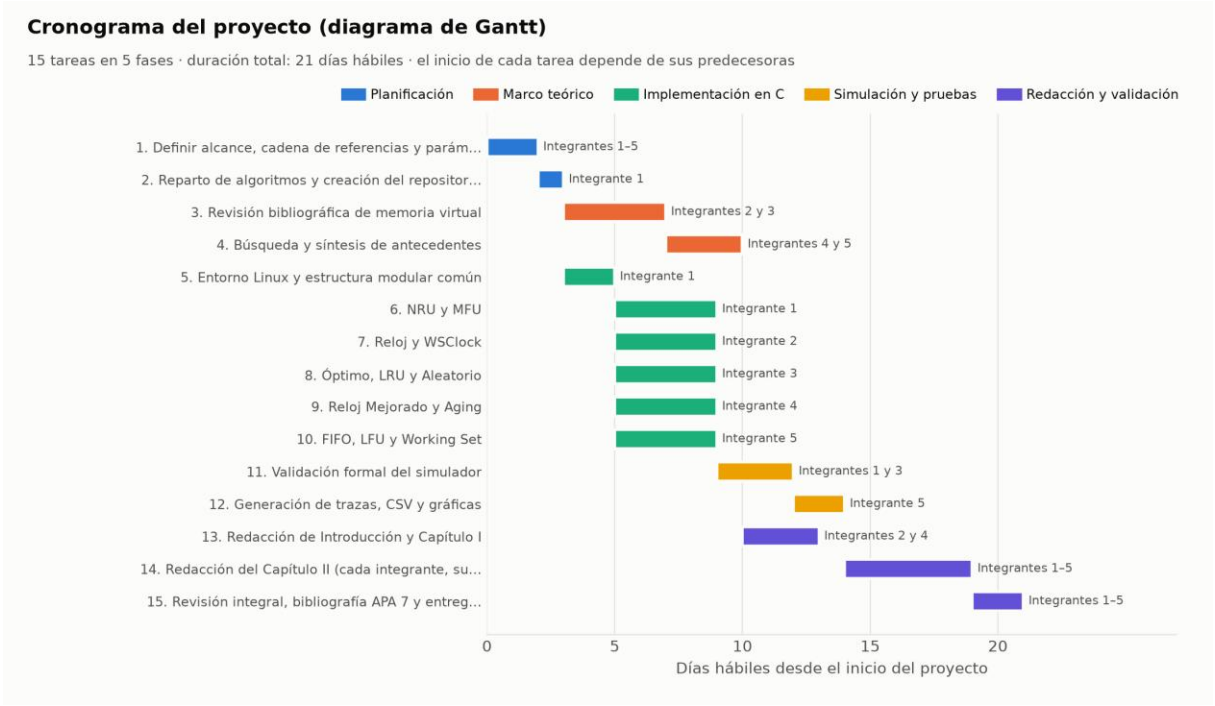
Cronograma del proyecto por fases, tareas, recursos y predecesoras

ID	Fase	Tarea	Días	Recursos	Pred.	Semana
1	Planificación	Definir alcance, cadena de referencias y parámetros	2	Integrantes 1–5	—	1
2	Planificación	Reparto de algoritmos y creación del repositorio	1	Integrante 1	1	1
3	Marco teórico	Revisión bibliográfica de memoria virtual	4	Integrantes 2 y 3	2	1–2
4	Marco teórico	Búsqueda y síntesis de antecedentes	3	Integrantes 4 y 5	3	2
5	Implementación en C	Entorno Linux y estructura modular común	2	Integrante 1	2	1
6	Implementación en C	NRU y MFU	4	Integrante 1	5	2
7	Implementación en C	Reloj y WSClock	4	Integrante 2	5	2
8	Implementación en C	Óptimo, LRU y Aleatorio	4	Integrante 3	5	2
9	Implementación en C	Reloj Mejorado y Aging	4	Integrante 4	5	2
10	Implementación en C	FIFO, LFU y Working Set	4	Integrante 5	5	2
11	Simulación y pruebas	Validación formal (referencia, invariantes, casos de borde)	3	Integrantes 1 y 3	6–10	3
12	Simulación y pruebas	Generación de trazas, CSV y gráficas	2	Integrante 5	11	3
13	Redacción y validación	Redacción de Introducción y Capítulo I	3	Integrantes 2 y 4	4	3
14	Redacción y validación	Redacción del Capítulo II (cada integrante, sus algoritmos)	5	Integrantes 1–5	12	4
15	Redacción y validación	Revisión integral, bibliografía APA 7 y entrega	2	Integrantes 1–5	13, 14	4

Nota. Integrante 1: Angel Ismael Alvarez Catunta; Integrante 2: Haider Alex Carpio Hermoza; Integrante 3: Jhamil Jhonatan Huahualuque Vargas; Integrante 4: Marcus Geuseppe Meza Zamalloa; Integrante 5: Elvis Raúl Suca Hancco.

Figura 1

Diagrama de Gantt del proyecto



Nota. Elaboración propia a partir de la Tabla 1.

CAPÍTULO II: MARCO TEÓRICO Y MODELADO DE LOS ALGORITMOS

2.1 Antecedentes internacionales

Antecedente internacional 1. Belady, L. A. (1966). *A study of replacement algorithms for a virtual-storage computer*. IBM Systems Journal.

Estudió el comportamiento de distintos algoritmos de reemplazo sobre trazas de programas reales en un computador con almacenamiento virtual de IBM. Definió el algoritmo **MIN** (óptimo), que expulsa la página cuya próxima referencia está más lejos, y lo usó como cota inferior para comparar estrategias realizables como la aleatoria y FIFO. Concluyó que la información sobre el uso reciente mejora significativamente las decisiones de reemplazo.

Relación con el proyecto: proporciona el algoritmo Óptimo que este proyecto utiliza como referencia (15 fallos) y el método de evaluación por simulación de trazas.

Antecedente internacional 2. Denning, P. J. (1968). *The working set model for program behavior*. Communications of the ACM.

Propuso el modelo del **conjunto de trabajo** $W(t, \tau)$: el conjunto de páginas referenciadas en la ventana de las últimas τ referencias. Demostró que un proceso solo progresa eficientemente si su conjunto de trabajo cabe en memoria y explicó la **hiperpaginación** como consecuencia de asignar menos marcos que los que exige la localidad del programa.

Relación con el proyecto: sustenta los algoritmos Working Set y WSClock implementados ($\tau = 4$) y la interpretación de la curva de fallos frente al número de marcos.

Antecedente internacional 3. Carr, R. W., & Hennessy, J. L. (1981). *WSCLOCK—A simple and effective algorithm for virtual memory management*. ACM SOSP.

Propusieron **WSClock**, que combina la estructura circular del algoritmo del reloj con el criterio del conjunto de trabajo y con la escritura anticipada de páginas modificadas. Mediante simulación mostraron que alcanza un rendimiento comparable al de políticas más costosas con una implementación sencilla.

Relación con el proyecto: WSClock fue, junto con LFU, el algoritmo realizable con menos fallos (19) y menos escrituras (4) en este proyecto, lo que concuerda con lo reportado por los autores.

Antecedente internacional 4. Megiddo, N., & Modha, D. S. (2003). *ARC: A self-tuning, low overhead replacement cache*. USENIX FAST.

Presentaron **ARC**, una política que mantiene dos listas, una para páginas usadas una vez (recencia) y otra para páginas usadas varias veces (frecuencia), y ajusta dinámicamente el espacio de cada una. En trazas reales superó a LRU con un costo de implementación bajo.

Relación con el proyecto: muestra que la combinación de **recencia y frecuencia** es una línea actual de investigación; en este proyecto, LFU (frecuencia) y LRU (recencia) obtuvieron 19 y 22 fallos, respectivamente.

2.2 Antecedentes nacionales

Antecedente nacional 1. Ruiz L., E., & Raffo L., E. (2003). *Simulación de la memoria cache*. Industrial Data, 6(2), 48–55. Universidad Nacional Mayor de San Marcos, Lima.

Desarrollaron un programa en C++ que simula una memoria caché modelada como una **cola FIFO**, con accesos generados mediante una distribución uniforme, para estimar el **porcentaje de aciertos** y el **tiempo medio de acceso**. Concluyeron que, para un tamaño dado de memoria principal, existe un **tamaño óptimo de caché** por encima del cual el porcentaje de aciertos ya no mejora, por lo que no conviene aumentarla indefinidamente.

Relación con el proyecto: comparte el enfoque de simulación en lenguaje C/C++, el uso de FIFO como política de reemplazo y las métricas de aciertos. Su conclusión es análoga a la observada en la curva de fallos de este proyecto: a partir de 7 marcos, agregar memoria ya no reduce los fallos.

2.3 Temas relacionados al proyecto con modelado de cada algoritmo

2.3.1 Fundamentos de la memoria virtual y condiciones del experimento

Paginación por demanda y fallo de página. La memoria lógica de un proceso se divide en **páginas** y la física en **marcos** del mismo tamaño. La tabla de páginas indica en qué marco está cada página y, mediante el bit de validez, si está en memoria. Al referenciar una página no válida se produce un **fallo de página**: el sistema operativo localiza la página en disco, la carga en un marco libre (o en el de una víctima), actualiza la tabla y reinicia la instrucción (Silberschatz et al., 2018).

El **tiempo efectivo de acceso** es EAT = (1 − p) × t_memoria + p × t_fallo, donde p es la tasa de fallos. Como t_fallo es miles de veces mayor que t_memoria, reducir p es el objetivo de todo algoritmo de reemplazo.

Localidad de referencia y bits R y M. Los programas presentan **localidad temporal** (lo usado recientemente se volverá a usar) y **localidad espacial** (se usan direcciones cercanas). Los algoritmos realizables explotan la localidad temporal para predecir el futuro a partir del pasado (Denning, 1970).

El hardware mantiene por página un **bit de referencia R**, que se activa en cada acceso, y un **bit de modificación M**, que se activa en cada escritura. Una página con M = 1 (sucia) debe escribirse en disco antes de reutilizar su marco, lo que duplica el costo de E/S de ese reemplazo.

Algoritmos de pila y anomalía de Belády. Un algoritmo es **de pila** si el conjunto de páginas en memoria con m marcos está siempre contenido en el conjunto con m + 1 marcos; en ese caso, más marcos nunca producen más fallos (Mattson et al., 1970). LRU y el Óptimo son algoritmos de pila; FIFO no lo es, y por ello puede presentar la **anomalía de Belády**: más fallos con más memoria (Belady et al., 1969).

Conjunto de trabajo e hiperpaginación. El **conjunto de trabajo** W(t, τ) contiene las páginas referenciadas en las últimas τ referencias. Si los marcos asignados son menos que |W|, el proceso falla continuamente y el sistema entra en **hiperpaginación** (Denning, 1968).

Condiciones del experimento. Todos los algoritmos se evaluaron con **3 marcos** inicialmente vacíos (las primeras cargas cuentan como fallo) y con la cadena oficial de 30 referencias, que contiene 7 páginas distintas (mínimo 7 fallos obligatorios). Cada referencia es una lectura (R) o una escritura (W) según el patrón de la Tabla 2. Si hay un marco libre se usa el de menor índice; los empates se resuelven como indica cada algoritmo o, en su defecto, a favor del marco de menor índice.

En las trazas, las columnas M1, M2 y M3 muestran el contenido de los marcos **después** de cada referencia; la página cargada en ese paso aparece en **negrita** y sombreada; la columna *Resultado* indica si hubo fallo o acierto; *Víctima* es la página expulsada, y la última columna muestra las variables internas del algoritmo.

Tabla 2

Cadena oficial de 30 referencias y tipo de operación

Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Página	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Operación	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R

Nota. R = lectura, W = escritura. 7 páginas distintas; 12 escrituras (40 %). Páginas de datos (0 y 2) se escriben con frecuencia; páginas de código (1, 5 y 7) solo se leen.

2.3.2 Algoritmo 1: FIFO (First-In, First-Out)

Responsable: Integrante 5 – Elvis Raúl Suca Hancco

a) Fundamento teórico y principio de operación

FIFO expulsa la página que lleva **más tiempo en memoria**, como una cola de llegada. No considera si la página se usa con frecuencia o no; un acierto no modifica el orden de la cola. Se implementa con un puntero circular al marco más antiguo (Silberschatz et al., 2018).

Tabla 3

Ventajas, desventajas, complejidad y anomalía de Belády de FIFO

Ventajas	Desventajas
• Muy simple de implementar y de bajo costo.	• Ignora el uso de las páginas: puede expulsar una página muy utilizada.
• No requiere apoyo especial del hardware.	• Presenta la anomalía de Belády.
Complejidad: O(1) por reemplazo con puntero circular (O(m) para buscar la página); espacio O(m).	**Anomalía de Belády: Sí.** Con la cadena 1,2,3,4,1,2,5,1,2,3,4,5 produce 9 fallos con 3 marcos y 10 con 4.

b) Modelado y traza completa paso a paso

La Tabla 4 muestra la ejecución de FIFO sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 4

Traza de FIFO con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`cola=[7]`
2	0	W	7	**0**	-	**Fallo**	-	`cola=[7,0]`
3	1	R	7	0	**1**	**Fallo**	-	`cola=[7,0,1]`
4	2	R	**2**	0	1	**Fallo**	7	`cola=[0,1,2]`
5	0	W	2	0	1	Acierto	-	`cola=[0,1,2]`
6	3	R	2	**3**	1	**Fallo**	0	`cola=[1,2,3]`
7	0	R	2	3	**0**	**Fallo**	1	`cola=[2,3,0]`
8	4	W	**4**	3	0	**Fallo**	2	`cola=[3,0,4]`
9	2	W	4	**2**	0	**Fallo**	3	`cola=[0,4,2]`
10	3	R	4	2	**3**	**Fallo**	0	`cola=[4,2,3]`
11	0	W	**0**	2	3	**Fallo**	4	`cola=[2,3,0]`
12	3	R	0	2	3	Acierto	-	`cola=[2,3,0]`
13	2	R	0	2	3	Acierto	-	`cola=[2,3,0]`
14	1	R	0	**1**	3	**Fallo**	2	`cola=[3,0,1]`
15	2	W	0	1	**2**	**Fallo**	3	`cola=[0,1,2]`
16	0	R	0	1	2	Acierto	-	`cola=[0,1,2]`
17	1	R	0	1	2	Acierto	-	`cola=[0,1,2]`
18	7	R	**7**	1	2	**Fallo**	0	`cola=[1,2,7]`
19	0	W	7	**0**	2	**Fallo**	1	`cola=[2,7,0]`
20	1	R	7	0	**1**	**Fallo**	2	`cola=[7,0,1]`
21	3	W	**3**	0	1	**Fallo**	7	`cola=[0,1,3]`
22	5	R	3	**5**	1	**Fallo**	0	`cola=[1,3,5]`
23	2	W	3	5	**2**	**Fallo**	1	`cola=[3,5,2]`
24	0	W	**0**	5	2	**Fallo**	3	`cola=[5,2,0]`
25	1	R	0	**1**	2	**Fallo**	5	`cola=[2,0,1]`
26	4	R	0	1	**4**	**Fallo**	2	`cola=[0,1,4]`
27	2	R	**2**	1	4	**Fallo**	0	`cola=[1,4,2]`
28	3	W	2	**3**	4	**Fallo**	1	`cola=[4,2,3]`
29	0	W	2	3	**0**	**Fallo**	4	`cola=[2,3,0]`
30	1	R	**1**	3	0	**Fallo**	2	`cola=[3,0,1]`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/01_fifo.txt`.

Resumen estadístico: 25 fallos de página, 5 aciertos, tasa de fallos 83.33 % (tasa de aciertos 16.67 %) y 9 escrituras a disco.

FIFO obtiene 25 fallos. Es especialmente desfavorable en el paso 6, donde expulsa la página 0 justo antes de que se vuelva a necesitar en el paso 7. La columna auxiliar muestra la cola, de la página más antigua a la más reciente.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/fifo.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 1`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * fifo.c - Algoritmo 1: FIFO (First-In, First-Out)
```

```
 * Responsable: Integrante 5 (Elvis Suca)
 *
 * Idea: se expulsa la página que lleva más tiempo en memoria.
 * Como los marcos vacíos se llenan en orden (M1, M2, M3) y cada reemplazo
 * ocurre en el marco más antiguo, basta un puntero circular que indica
 * "el marco que llegó primero". Un acierto NO modifica la cola.
 *
 * Variables auxiliares mostradas: cola de llegada (más antigua primero).
 *
 * Este módulo es la PLANTILLA de referencia para los demás algoritmos.
 */
#include "../algoritmos.h"

/* Arma la cola de llegada empezando por el marco más antiguo */
static const char *cola_fifo(char *buf, size_t len, const int marcos[], int m, int puntero)
{
    int cola[MAX_MARCOS];
    int cant = 0;

    for (int k = 0; k < m; k++) {
        int idx = (puntero + k) % m;
        if (marcos[idx] != MARCO_VACIO)
            cola[cant++] = marcos[idx];
    }
    return fmt_arreglo(buf, len, cola, cant);
}

void simular_fifo(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int puntero = 0;            /* marco con la página más antigua */
    char buf[AUX_LEN];

    resultado_iniciar(r, "FIFO", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int marco, victima = -1;

        if (buscar_pagina(marcos, cfg->marcos, pagina) >= 0) {
            /* Acierto: la cola no cambia */
            registrar_paso(r, cfg, i, marcos, -1, -1, "cola=%s",
                           cola_fifo(buf, sizeof buf, marcos, cfg->marcos, puntero));
            continue;
        }

        /* Fallo de página */
        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = puntero;                        /* el más antiguo */
            victima = marcos[marco];
            puntero = (puntero + 1) % cfg->marcos;
        }
        marcos[marco] = pagina;

        registrar_paso(r, cfg, i, marcos, marco, victima, "cola=%s",
                       cola_fifo(buf, sizeof buf, marcos, cfg->marcos, puntero));
    }
}
```

d) Citas bibliográficas

Fundamentado en Silberschatz et al. (2018); Belady et al. (1969).

2.3.3 Algoritmo 2: Óptimo (OPT / MIN de Belády)

Responsable: Integrante 3 – Jhamil Jhonatan Huahualuque Vargas

a) Fundamento teórico y principio de operación

El algoritmo óptimo expulsa la página cuya **próxima referencia está más lejos en el futuro**, o que no se volverá a usar. Produce el menor número de fallos posible para cualquier cadena y número de marcos (Belady, 1966). Como exige conocer la cadena completa por adelantado, no puede implementarse en un sistema operativo real; se usa como **cota inferior** para evaluar los demás algoritmos.

Tabla 5

Ventajas, desventajas, complejidad y anomalía de Belády de Óptimo

Ventajas	Desventajas
• Mínimo número de fallos posible.	• No es realizable: requiere conocer el futuro.
• Es un algoritmo de pila: no presenta la anomalía de Belády.	• Costoso de calcular: debe recorrer la cadena futura.
Complejidad: O(m·n) por reemplazo con búsqueda directa en la cadena futura; espacio O(n).	**Anomalía de Belády: No** (algoritmo de pila). Con la cadena de Belády: 7 fallos con 3 marcos y 6 con 4.

b) Modelado y traza completa paso a paso

La Tabla 6 muestra la ejecución de Óptimo sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 6

Traza de Óptimo con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`prox=[18,-,-]`
2	0	W	7	**0**	-	**Fallo**	-	`prox=[18,5,-]`
3	1	R	7	0	**1**	**Fallo**	-	`prox=[18,5,14]`
4	2	R	**2**	0	1	**Fallo**	7	`prox=[18,5,14] -> sale la mas lejana`
5	0	W	2	0	1	Acierto	-	`prox=[9,7,14]`
6	3	R	2	0	**3**	**Fallo**	1	`prox=[9,7,14] -> sale la mas lejana`
7	0	R	2	0	3	Acierto	-	`prox=[9,11,10]`
8	4	W	2	**4**	3	**Fallo**	0	`prox=[9,11,10] -> sale la mas lejana`
9	2	W	2	4	3	Acierto	-	`prox=[13,26,10]`
10	3	R	2	4	3	Acierto	-	`prox=[13,26,12]`
11	0	W	2	**0**	3	**Fallo**	4	`prox=[13,26,12] -> sale la mas lejana`
12	3	R	2	0	3	Acierto	-	`prox=[13,16,21]`
13	2	R	2	0	3	Acierto	-	`prox=[15,16,21]`
14	1	R	2	0	**1**	**Fallo**	3	`prox=[15,16,21] -> sale la mas lejana`
15	2	W	2	0	1	Acierto	-	`prox=[23,16,17]`
16	0	R	2	0	1	Acierto	-	`prox=[23,19,17]`
17	1	R	2	0	1	Acierto	-	`prox=[23,19,20]`
18	7	R	**7**	0	1	**Fallo**	2	`prox=[23,19,20] -> sale la mas lejana`
19	0	W	7	0	1	Acierto	-	`prox=[--,24,20]`
20	1	R	7	0	1	Acierto	-	`prox=[--,24,25]`
21	3	W	**3**	0	1	**Fallo**	7	`prox=[--,24,25] -> sale la mas lejana`
22	5	R	**5**	0	1	**Fallo**	3	`prox=[28,24,25] -> sale la mas lejana`
23	2	W	**2**	0	1	**Fallo**	5	`prox=[--,24,25] -> sale la mas lejana`
24	0	W	2	0	1	Acierto	-	`prox=[27,29,25]`
25	1	R	2	0	1	Acierto	-	`prox=[27,29,30]`
26	4	R	2	0	**4**	**Fallo**	1	`prox=[27,29,30] -> sale la mas lejana`
27	2	R	2	0	4	Acierto	-	`prox=[--,29,--]`
28	3	W	**3**	0	4	**Fallo**	2	`prox=[--,29,--] -> sale la mas lejana`
29	0	W	3	0	4	Acierto	-	`prox=[--,--,--]`
30	1	R	**1**	0	4	**Fallo**	3	`prox=[--,--,--] -> sale la mas lejana`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/02_optimo.txt`.

Resumen estadístico: 15 fallos de página, 15 aciertos, tasa de fallos 50.00 % (tasa de aciertos 50.00 %) y 6 escrituras a disco.

Obtiene 15 fallos (50 %). La columna *prox* indica el paso en que se volverá a referenciar la página de cada marco ("--" significa nunca); en un reemplazo se muestran los valores con que se decidió. Por ejemplo, en el paso 18 se expulsa la página 2, que no se vuelve a usar hasta el paso 23.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/optimo.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 2`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * optimo.c - Algoritmo 2: Óptimo (OPT / MIN de Belády)
```

```
 * Responsable: Integrante 3 (Jhamil Huahualuque)
 *
 * Idea: se expulsa la página cuyo PRÓXIMO uso está más lejos en el futuro
 * (o que no se volverá a usar). Requiere conocer toda la cadena por
 * adelantado, por eso no es implementable en un SO real; sirve como
 * cota mínima de fallos para comparar a los demás algoritmos.
 *
 * Empate (varias páginas que no se vuelven a usar): menor índice de marco.
 * Variables auxiliares: paso del próximo uso de cada marco ("--" = nunca).
 *   En un reemplazo se muestran los valores con los que se decidió.
 */
#include "../algoritmos.h"

#define NUNCA 1000000   /* distancia "infinita": la página no se vuelve a usar */

/* Paso (0-based) del próximo uso de 'pagina' después de 'desde', o NUNCA */
static int proximo_uso(const Config *cfg, int pagina, int desde)
{
    for (int j = desde + 1; j < cfg->n; j++)
        if (cfg->refs[j] == pagina)
            return j;
    return NUNCA;
}

/* Texto "prox=[5,14,--]" con pasos en base 1 */
static const char *fmt_proximos(char *buf, size_t len, const Config *cfg,
                                const int marcos[], int i)
{
    size_t u = (size_t)snprintf(buf, len, "prox=[");
    for (int k = 0; k < cfg->marcos && u < len; k++) {
        const char *sep = (k + 1 < cfg->marcos) ? "," : "]";
        int prox = marcos[k] == MARCO_VACIO ? NUNCA : proximo_uso(cfg, marcos[k], i);
        if (marcos[k] == MARCO_VACIO)
            u += (size_t)snprintf(buf + u, len - u, "-%s", sep);
        else if (prox == NUNCA)
            u += (size_t)snprintf(buf + u, len - u, "--%s", sep);
        else
            u += (size_t)snprintf(buf + u, len - u, "%d%s", prox + 1, sep);
    }
    return buf;
}

void simular_optimo(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    char buf[AUX_LEN];

    resultado_iniciar(r, "Optimo (OPT)", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int marco, victima = -1;

        if (buscar_pagina(marcos, cfg->marcos, pagina) >= 0) {
            registrar_paso(r, cfg, i, marcos, -1, -1, "%s",
                           fmt_proximos(buf, sizeof buf, cfg, marcos, i));
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            int mas_lejano = -1;
            /* Decidir con el estado ANTES del reemplazo */
            fmt_proximos(buf, sizeof buf, cfg, marcos, i);
            for (int k = 0; k < cfg->marcos; k++) {
                int prox = proximo_uso(cfg, marcos[k], i);
                if (prox > mas_lejano) {        /* '>' estricto: empate -> menor índice */
                    mas_lejano = prox;
                    marco = k;
                }
            }
            victima = marcos[marco];
```

```
            marcos[marco] = pagina;
            registrar_paso(r, cfg, i, marcos, marco, victima, "%s -> sale la mas lejana", buf);
            continue;
        }

        marcos[marco] = pagina;
        registrar_paso(r, cfg, i, marcos, marco, victima, "%s",
                       fmt_proximos(buf, sizeof buf, cfg, marcos, i));
    }
}
```

d) Citas bibliográficas

Fundamentado en Belady (1966); Mattson et al. (1970).

2.3.4 Algoritmo 3: LRU (Least Recently Used)

Responsable: Integrante 3 – Jhamil Jhonatan Huahualuque Vargas

a) Fundamento teórico y principio de operación

LRU expulsa la página que lleva **más tiempo sin usarse**, bajo el supuesto de localidad temporal: el pasado reciente predice el futuro cercano. Se implementó con una marca de tiempo del último uso por marco; la víctima es la marca menor (Silberschatz et al., 2018).

Tabla 7

Ventajas, desventajas, complejidad y anomalía de Belády de LRU

Ventajas	Desventajas
• Buena aproximación del Óptimo en programas con localidad.	• Su implementación exacta exige actualizar una marca en cada acceso a memoria, lo que es costoso en hardware.
• Es un algoritmo de pila.	• Falla siempre ante recorridos cíclicos mayores que la memoria.
Complejidad: O(m) por referencia con marcas de tiempo (O(1) con lista doblemente enlazada y tabla hash); espacio O(m).	**Anomalía de Belády: No** (algoritmo de pila). Con la cadena de Belády: 10 fallos con 3 marcos y 8 con 4.

b) Modelado y traza completa paso a paso

La Tabla 8 muestra la ejecución de LRU sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 8

Traza de LRU con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`ult=[1,-,-]`
2	0	W	7	**0**	-	**Fallo**	-	`ult=[1,2,-]`
3	1	R	7	0	**1**	**Fallo**	-	`ult=[1,2,3]`
4	2	R	**2**	0	1	**Fallo**	7	`ult=[4,2,3]`
5	0	W	2	0	1	Acierto	-	`ult=[4,5,3]`
6	3	R	2	0	**3**	**Fallo**	1	`ult=[4,5,6]`
7	0	R	2	0	3	Acierto	-	`ult=[4,7,6]`
8	4	W	**4**	0	3	**Fallo**	2	`ult=[8,7,6]`
9	2	W	4	0	**2**	**Fallo**	3	`ult=[8,7,9]`
10	3	R	4	**3**	2	**Fallo**	0	`ult=[8,10,9]`
11	0	W	**0**	3	2	**Fallo**	4	`ult=[11,10,9]`
12	3	R	0	3	2	Acierto	-	`ult=[11,12,9]`
13	2	R	0	3	2	Acierto	-	`ult=[11,12,13]`
14	1	R	**1**	3	2	**Fallo**	0	`ult=[14,12,13]`
15	2	W	1	3	2	Acierto	-	`ult=[14,12,15]`
16	0	R	1	**0**	2	**Fallo**	3	`ult=[14,16,15]`
17	1	R	1	0	2	Acierto	-	`ult=[17,16,15]`
18	7	R	1	0	**7**	**Fallo**	2	`ult=[17,16,18]`
19	0	W	1	0	7	Acierto	-	`ult=[17,19,18]`
20	1	R	1	0	7	Acierto	-	`ult=[20,19,18]`
21	3	W	1	0	**3**	**Fallo**	7	`ult=[20,19,21]`
22	5	R	1	**5**	3	**Fallo**	0	`ult=[20,22,21]`
23	2	W	**2**	5	3	**Fallo**	1	`ult=[23,22,21]`
24	0	W	2	5	**0**	**Fallo**	3	`ult=[23,22,24]`
25	1	R	2	**1**	0	**Fallo**	5	`ult=[23,25,24]`
26	4	R	**4**	1	0	**Fallo**	2	`ult=[26,25,24]`
27	2	R	4	1	**2**	**Fallo**	0	`ult=[26,25,27]`
28	3	W	4	**3**	2	**Fallo**	1	`ult=[26,28,27]`
29	0	W	**0**	3	2	**Fallo**	4	`ult=[29,28,27]`
30	1	R	0	3	**1**	**Fallo**	2	`ult=[29,28,30]`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/03_lru.txt`.

Resumen estadístico: 22 fallos de página, 8 aciertos, tasa de fallos 73.33 % (tasa de aciertos 26.67 %) y 8 escrituras a disco.

Obtiene 22 fallos. La columna *ult* muestra el paso del último uso de la página de cada marco; la víctima es siempre la de menor valor.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/lru.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 3`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * lru.c - Algoritmo 3: LRU (Least Recently Used)
 * Responsable: Integrante 3 (Jhamil Huahualuque)
 *
```

```
 * Idea: se expulsa la página que lleva MÁS TIEMPO sin usarse.
 * Implementación con "contador/marca de tiempo" (Silberschatz): cada marco
 * guarda el paso de su último uso; la víctima es la marca más pequeña.
 * Es un algoritmo de pila: no sufre la anomalía de Belády.
 *
 * Variables auxiliares: ult = paso (base 1) del último uso de cada marco.
 */
#include "../algoritmos.h"

void simular_lru(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int ultimo[MAX_MARCOS];     /* paso del último uso, base 1 */
    char buf[AUX_LEN];

    resultado_iniciar(r, "LRU", cfg);
    marcos_vaciar(marcos, cfg->marcos);
    marcos_vaciar(ultimo, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1;

        if (idx >= 0) {
            ultimo[idx] = i + 1;
            registrar_paso(r, cfg, i, marcos, -1, -1, "ult=%s",
                           fmt_arreglo(buf, sizeof buf, ultimo, cfg->marcos));
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = 0;
            for (int k = 1; k < cfg->marcos; k++)
                if (ultimo[k] < ultimo[marco])
                    marco = k;
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        ultimo[marco] = i + 1;
        registrar_paso(r, cfg, i, marcos, marco, victima, "ult=%s",
                       fmt_arreglo(buf, sizeof buf, ultimo, cfg->marcos));
    }
}
```

d) Citas bibliográficas

Fundamentado en Silberschatz et al. (2018); Mattson et al. (1970).

2.3.5 Algoritmo 4: Reloj o Segunda Oportunidad (Clock)

Responsable: Integrante 2 – Haider Alex Carpio Hermoza

a) Fundamento teórico y principio de operación

Es una variante de FIFO con un **bit de referencia R** y un puntero circular. Si la página bajo el puntero tiene R = 1, se le da una segunda oportunidad (R = 0) y el puntero avanza; si tiene R = 0, se expulsa. Fue introducido en el sistema Multics (Corbató, 1969).

Tabla 9

Ventajas, desventajas, complejidad y anomalía de Belády de Reloj o Segunda Oportunidad

Ventajas	Desventajas
• Aproxima LRU con un solo bit por página.	• Si todas las páginas tienen R = 1, degenera en FIFO.
• Implementación eficiente, usada en sistemas reales.	• Puede presentar la anomalía de Belády.
Complejidad: O(1) amortizado y O(m) en el peor caso (una vuelta completa); espacio: 1 bit por marco.	**Anomalía de Belády: Sí.** Con la cadena de Belády: 9 fallos con 3 marcos y 10 con 4.

b) Modelado y traza completa paso a paso

La Tabla 10 muestra la ejecución de Reloj o Segunda Oportunidad sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 10

Traza de Reloj o Segunda Oportunidad con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`R=[1,0,0] ptr=M1`
2	0	W	7	**0**	-	**Fallo**	-	`R=[1,1,0] ptr=M1`
3	1	R	7	0	**1**	**Fallo**	-	`R=[1,1,1] ptr=M1`
4	2	R	**2**	0	1	**Fallo**	7	`R=[1,0,0] ptr=M2 (2das oport.: 3)`
5	0	W	2	0	1	Acierto	-	`R=[1,1,0] ptr=M2`
6	3	R	2	0	**3**	**Fallo**	1	`R=[1,0,1] ptr=M1 (2das oport.: 1)`
7	0	R	2	0	3	Acierto	-	`R=[1,1,1] ptr=M1`
8	4	W	**4**	0	3	**Fallo**	2	`R=[1,0,0] ptr=M2 (2das oport.: 3)`
9	2	W	4	**2**	3	**Fallo**	0	`R=[1,1,0] ptr=M3 (2das oport.: 0)`
10	3	R	4	2	3	Acierto	-	`R=[1,1,1] ptr=M3`
11	0	W	4	2	**0**	**Fallo**	3	`R=[0,0,1] ptr=M1 (2das oport.: 3)`
12	3	R	**3**	2	0	**Fallo**	4	`R=[1,0,1] ptr=M2 (2das oport.: 0)`
13	2	R	3	2	0	Acierto	-	`R=[1,1,1] ptr=M2`
14	1	R	3	**1**	0	**Fallo**	2	`R=[0,1,0] ptr=M3 (2das oport.: 3)`
15	2	W	3	1	**2**	**Fallo**	0	`R=[0,1,1] ptr=M1 (2das oport.: 0)`
16	0	R	**0**	1	2	**Fallo**	3	`R=[1,1,1] ptr=M2 (2das oport.: 0)`
17	1	R	0	1	2	Acierto	-	`R=[1,1,1] ptr=M2`
18	7	R	0	**7**	2	**Fallo**	1	`R=[0,1,0] ptr=M3 (2das oport.: 3)`
19	0	W	0	7	2	Acierto	-	`R=[1,1,0] ptr=M3`
20	1	R	0	7	**1**	**Fallo**	2	`R=[1,1,1] ptr=M1 (2das oport.: 0)`
21	3	W	**3**	7	1	**Fallo**	0	`R=[1,0,0] ptr=M2 (2das oport.: 3)`
22	5	R	3	**5**	1	**Fallo**	7	`R=[1,1,0] ptr=M3 (2das oport.: 0)`
23	2	W	3	5	**2**	**Fallo**	1	`R=[1,1,1] ptr=M1 (2das oport.: 0)`
24	0	W	**0**	5	2	**Fallo**	3	`R=[1,0,0] ptr=M2 (2das oport.: 3)`
25	1	R	0	**1**	2	**Fallo**	5	`R=[1,1,0] ptr=M3 (2das oport.: 0)`
26	4	R	0	1	**4**	**Fallo**	2	`R=[1,1,1] ptr=M1 (2das oport.: 0)`
27	2	R	**2**	1	4	**Fallo**	0	`R=[1,0,0] ptr=M2 (2das oport.: 3)`
28	3	W	2	**3**	4	**Fallo**	1	`R=[1,1,0] ptr=M3 (2das oport.: 0)`
29	0	W	2	3	**0**	**Fallo**	4	`R=[1,1,1] ptr=M1 (2das oport.: 0)`
30	1	R	**1**	3	0	**Fallo**	2	`R=[1,0,0] ptr=M2 (2das oport.: 3)`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/04_reloj.txt`.

Resumen estadístico: 24 fallos de página, 6 aciertos, tasa de fallos 80.00 % (tasa de aciertos 20.00 %) y 9 escrituras a disco.

Obtiene 24 fallos, cerca de FIFO. En varios pasos todas las páginas tienen R = 1 (por ejemplo, el paso 18) y el puntero da una vuelta completa antes de elegir la víctima: en esos casos se comporta como FIFO.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/reloj.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 4`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * reloj.c - Algoritmo 4: Reloj / Segunda Oportunidad (Clock)
 * Responsable: Integrante 2 (Haider Carpio)
 *
 * Idea: FIFO circular con bit de referencia R. El puntero (manecilla)
 * recorre los marcos: si R = 1 se le da una "segunda oportunidad"
 * (R = 0 y avanza); si R = 0, esa página es la víctima.
 * Al cargar o en un acierto: R = 1. Tras reemplazar, el puntero avanza.
 * El puntero empieza en M1 y no se mueve mientras se llenan marcos libres.
 *
 * Variables auxiliares: bits R de cada marco y posición del puntero.
 */
#include "../algoritmos.h"

void simular_reloj(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int puntero = 0;
    char buf[AUX_LEN];

    resultado_iniciar(r, "Reloj (Clock)", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1, segundas = 0;

        if (idx >= 0) {
            bit_r[idx] = 1;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s ptr=M%d",
                           fmt_arreglo(buf, sizeof buf, bit_r, cfg->marcos), puntero + 1);
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            while (bit_r[puntero] == 1) {       /* segunda oportunidad */
                bit_r[puntero] = 0;
                puntero = (puntero + 1) % cfg->marcos;
                segundas++;
            }
            marco = puntero;
            victima = marcos[marco];
            puntero = (puntero + 1) % cfg->marcos;
        }
        marcos[marco] = pagina;
        bit_r[marco] = 1;
```

```
        if (victima >= 0)
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s ptr=M%d (2das oport.: %d)",
                           fmt_arreglo(buf, sizeof buf, bit_r, cfg->marcos), puntero + 1, segundas);
        else
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s ptr=M%d",
                           fmt_arreglo(buf, sizeof buf, bit_r, cfg->marcos), puntero + 1);
    }
}
```

d) Citas bibliográficas

Fundamentado en Corbató (1969); Tanenbaum y Bos (2015).

2.3.6 Algoritmo 5: NRU (Not Recently Used)

Responsable: Integrante 1 – Angel Ismael Alvarez Catunta

a) Fundamento teórico y principio de operación

NRU clasifica las páginas según sus bits (R, M) en cuatro clases: 0 = (0,0), 1 = (0,1), 2 = (1,0) y 3 = (1,1), y expulsa una página de la clase más baja. Una interrupción de reloj periódica pone los bits R en 0, de modo que R indica uso **reciente** (Tanenbaum & Bos, 2015). En este proyecto el periodo es de 5 tics y los empates se resuelven por antigüedad (FIFO).

Tabla 11

Ventajas, desventajas, complejidad y anomalía de Belády de NRU

Ventajas	Desventajas
• Simple y de bajo costo.	• Solo distingue cuatro clases: decisión de grano grueso.
• Considera el costo de escritura (prefiere páginas limpias).	• Depende del periodo de la interrupción de reloj.
Complejidad: O(m) por reemplazo (revisión de toda la tabla); espacio: 2 bits por marco.	**Anomalía de Belády:** Posible, pues no es un algoritmo de pila; no se observó en las pruebas realizadas.

b) Modelado y traza completa paso a paso

La Tabla 12 muestra la ejecución de NRU sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 12

Traza de NRU con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`R=[1,0,0] M=[0,0,0]`
2	0	W	7	**0**	-	**Fallo**	-	`R=[1,1,0] M=[0,1,0]`
3	1	R	7	0	**1**	**Fallo**	-	`R=[1,1,1] M=[0,1,0]`
4	2	R	**2**	0	1	**Fallo**	7	`R=[1,1,1] M=[0,1,0] clase=2`
5	0	W	2	0	1	Acierto	-	`R=[1,1,1] M=[0,1,0]`
6	3	R	2	0	**3**	**Fallo**	1	`R=[0,0,1] M=[0,1,0] clase=0 [reloj: R<-0]`
7	0	R	2	0	3	Acierto	-	`R=[0,1,1] M=[0,1,0]`
8	4	W	**4**	0	3	**Fallo**	2	`R=[1,1,1] M=[1,1,0] clase=0`
9	2	W	4	0	**2**	**Fallo**	3	`R=[1,1,1] M=[1,1,1] clase=2`
10	3	R	4	**3**	2	**Fallo**	0	`R=[1,1,1] M=[1,0,1] clase=3`
11	0	W	4	**0**	2	**Fallo**	3	`R=[0,1,0] M=[1,1,1] clase=0 [reloj: R<-0]`
12	3	R	**3**	0	2	**Fallo**	4	`R=[1,1,0] M=[0,1,1] clase=1`
13	2	R	3	0	2	Acierto	-	`R=[1,1,1] M=[0,1,1]`
14	1	R	**1**	0	2	**Fallo**	3	`R=[1,1,1] M=[0,1,1] clase=2`
15	2	W	1	0	2	Acierto	-	`R=[1,1,1] M=[0,1,1]`
16	0	R	1	0	2	Acierto	-	`R=[0,1,0] M=[0,1,1] [reloj: R<-0]`
17	1	R	1	0	2	Acierto	-	`R=[1,1,0] M=[0,1,1]`
18	7	R	1	0	**7**	**Fallo**	2	`R=[1,1,1] M=[0,1,0] clase=1`
19	0	W	1	0	7	Acierto	-	`R=[1,1,1] M=[0,1,0]`
20	1	R	1	0	7	Acierto	-	`R=[1,1,1] M=[0,1,0]`
21	3	W	**3**	0	7	**Fallo**	1	`R=[1,0,0] M=[1,1,0] clase=0 [reloj: R<-0]`
22	5	R	3	0	**5**	**Fallo**	7	`R=[1,0,1] M=[1,1,0] clase=0`
23	2	W	3	**2**	5	**Fallo**	0	`R=[1,1,1] M=[1,1,0] clase=1`
24	0	W	3	2	**0**	**Fallo**	5	`R=[1,1,1] M=[1,1,1] clase=2`
25	1	R	**1**	2	0	**Fallo**	3	`R=[1,1,1] M=[0,1,1] clase=3`
26	4	R	**4**	2	0	**Fallo**	1	`R=[1,0,0] M=[0,1,1] clase=0 [reloj: R<-0]`
27	2	R	4	2	0	Acierto	-	`R=[1,1,0] M=[0,1,1]`
28	3	W	4	2	**3**	**Fallo**	0	`R=[1,1,1] M=[0,1,1] clase=1`
29	0	W	**0**	2	3	**Fallo**	4	`R=[1,1,1] M=[1,1,1] clase=2`
30	1	R	0	**1**	3	**Fallo**	2	`R=[1,1,1] M=[1,0,1] clase=3`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/05_nru.txt`.

Resumen estadístico: 21 fallos de página, 9 aciertos, tasa de fallos 70.00 % (tasa de aciertos 30.00 %) y 7 escrituras a disco.

Obtiene 21 fallos y 7 escrituras. La marca [reloj: R<-0] indica los pasos (6, 11, 16, 21 y 26) en que la interrupción limpia los bits R. En el paso 21, las páginas 1 y 7 eran de clase 0 y se expulsó la 1, que había llegado antes.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/nru.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 5`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * nru.c - Algoritmo 5: NRU (Not Recently Used)
```

```
 * Responsable: Integrante 1 (Angel Alvarez)
 *
 * Idea (Tanenbaum 3.4.2): cada página tiene bits R (referenciada) y
 * M (modificada). Una interrupción de reloj cada PERIODO_RELOJ tics pone
 * todos los R en 0. Las páginas se agrupan en 4 clases:
 *     clase 0 = (R=0,M=0)  no usada, limpia     <- mejor víctima
 *     clase 1 = (R=0,M=1)  no usada, sucia
 *     clase 2 = (R=1,M=0)  usada, limpia
 *     clase 3 = (R=1,M=1)  usada, sucia
 * Se expulsa una página de la clase más baja; empate: la más antigua (FIFO).
 * A diferencia del Reloj Mejorado, revisa TODA la tabla (sin puntero).
 *
 * Variables auxiliares: bits R y M, y aviso cuando el reloj limpia R.
 */
#include "../algoritmos.h"

void simular_nru(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int bit_m[MAX_MARCOS] = {0};
    int llegada[MAX_MARCOS] = {0};
    char br[AUX_LEN / 3], bm[AUX_LEN / 3];

    resultado_iniciar(r, "NRU", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int w = es_escritura(cfg, i);
        int reset = (i > 0 && i % PERIODO_RELOJ == 0);
        int idx, marco, victima = -1, clase_victima = -1;

        /* Interrupción de reloj: limpia los bits R al inicio del tic */
        if (reset)
            for (int k = 0; k < cfg->marcos; k++)
                bit_r[k] = 0;

        idx = buscar_pagina(marcos, cfg->marcos, pagina);
        if (idx >= 0) {
            bit_r[idx] = 1;
            bit_m[idx] |= w;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s M=%s%s",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           reset ? " [reloj: R<-0]" : "");
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            int mejor = 0;
            for (int k = 1; k < cfg->marcos; k++) {
                int ck = 2 * bit_r[k] + bit_m[k];
                int cm = 2 * bit_r[mejor] + bit_m[mejor];
                if (ck < cm || (ck == cm && llegada[k] < llegada[mejor]))
                    mejor = k;
            }
            marco = mejor;
            clase_victima = 2 * bit_r[marco] + bit_m[marco];
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        bit_r[marco] = 1;
        bit_m[marco] = w;
        llegada[marco] = i;

        if (victima >= 0)
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s clase=%d%s",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           clase_victima, reset ? " [reloj: R<-0]" : "");
        else
```

```
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s%s",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           reset ? " [reloj: R<-0]" : "");
    }
}
```

d) Citas bibliográficas

Fundamentado en Tanenbaum y Bos (2015).

2.3.7 Algoritmo 6: LFU (Least Frequently Used)

Responsable: Integrante 5 – Elvis Raúl Suca Hancco

a) Fundamento teórico y principio de operación

LFU mantiene un **contador de usos** por página y expulsa la de menor contador. El contador se reinicia en 1 al cargar la página; los empates se resuelven por antigüedad (Silberschatz et al., 2018).

Tabla 13

Ventajas, desventajas, complejidad y anomalía de Belády de LFU

Ventajas	Desventajas
• Protege las páginas usadas con mucha frecuencia.	• Una página muy usada en el pasado puede quedar en memoria aunque ya no se necesite.
• Simple de implementar.	• No es algoritmo de pila.
Complejidad: O(m) por reemplazo (O(log m) con un montículo); espacio: un contador por marco.	**Anomalía de Belády: Sí, posible:** en la búsqueda aleatoria se encontró al menos un caso (9 → 11 fallos al pasar de 3 a 4 marcos).

b) Modelado y traza completa paso a paso

La Tabla 14 muestra la ejecución de LFU sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 14

Traza de LFU con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`frec=[1,0,0]`
2	0	W	7	**0**	-	**Fallo**	-	`frec=[1,1,0]`
3	1	R	7	0	**1**	**Fallo**	-	`frec=[1,1,1]`
4	2	R	**2**	0	1	**Fallo**	7	`frec=[1,1,1]`
5	0	W	2	0	1	Acierto	-	`frec=[1,2,1]`
6	3	R	2	0	**3**	**Fallo**	1	`frec=[1,2,1]`
7	0	R	2	0	3	Acierto	-	`frec=[1,3,1]`
8	4	W	**4**	0	3	**Fallo**	2	`frec=[1,3,1]`
9	2	W	4	0	**2**	**Fallo**	3	`frec=[1,3,1]`
10	3	R	**3**	0	2	**Fallo**	4	`frec=[1,3,1]`
11	0	W	3	0	2	Acierto	-	`frec=[1,4,1]`
12	3	R	3	0	2	Acierto	-	`frec=[2,4,1]`
13	2	R	3	0	2	Acierto	-	`frec=[2,4,2]`
14	1	R	3	0	**1**	**Fallo**	2	`frec=[2,4,1]`
15	2	W	3	0	**2**	**Fallo**	1	`frec=[2,4,1]`
16	0	R	3	0	2	Acierto	-	`frec=[2,5,1]`
17	1	R	3	0	**1**	**Fallo**	2	`frec=[2,5,1]`
18	7	R	3	0	**7**	**Fallo**	1	`frec=[2,5,1]`
19	0	W	3	0	7	Acierto	-	`frec=[2,6,1]`
20	1	R	3	0	**1**	**Fallo**	7	`frec=[2,6,1]`
21	3	W	3	0	1	Acierto	-	`frec=[3,6,1]`
22	5	R	3	0	**5**	**Fallo**	1	`frec=[3,6,1]`
23	2	W	3	0	**2**	**Fallo**	5	`frec=[3,6,1]`
24	0	W	3	0	2	Acierto	-	`frec=[3,7,1]`
25	1	R	3	0	**1**	**Fallo**	2	`frec=[3,7,1]`
26	4	R	3	0	**4**	**Fallo**	1	`frec=[3,7,1]`
27	2	R	3	0	**2**	**Fallo**	4	`frec=[3,7,1]`
28	3	W	3	0	2	Acierto	-	`frec=[4,7,1]`
29	0	W	3	0	2	Acierto	-	`frec=[4,8,1]`
30	1	R	3	0	**1**	**Fallo**	2	`frec=[4,8,1]`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/06_lfu.txt`.

Resumen estadístico: 19 fallos de página, 11 aciertos, tasa de fallos 63.33 % (tasa de aciertos 36.67 %) y 4 escrituras a disco.

Obtiene 19 fallos y solo 4 escrituras, el mejor resultado realizable junto con WSClock. La página 0, la más referenciada (8 veces) y la más escrita, acumula un contador alto y, desde que entra en el paso 2, nunca es expulsada.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/lfu.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 6`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * lfu.c - Algoritmo 6: LFU (Least Frequently Used)
```

```
 * Responsable: Integrante 5 (Elvis Suca)
 *
 * Idea: cada página en memoria lleva un contador de usos; se expulsa la
 * de MENOR contador. El contador se reinicia a 1 al cargar la página
 * (no se guarda historial de páginas que están fuera de memoria).
 * Empate: la que llegó primero (FIFO).
 *
 * Variables auxiliares: contador de frecuencia de cada marco.
 */
#include "../algoritmos.h"

void simular_lfu(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int frec[MAX_MARCOS] = {0};
    int llegada[MAX_MARCOS] = {0};
    char buf[AUX_LEN];

    resultado_iniciar(r, "LFU", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1;

        if (idx >= 0) {
            frec[idx]++;
            registrar_paso(r, cfg, i, marcos, -1, -1, "frec=%s",
                           fmt_arreglo(buf, sizeof buf, frec, cfg->marcos));
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = 0;
            for (int k = 1; k < cfg->marcos; k++)
                if (frec[k] < frec[marco] ||
                    (frec[k] == frec[marco] && llegada[k] < llegada[marco]))
                    marco = k;
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        frec[marco] = 1;
        llegada[marco] = i;
        registrar_paso(r, cfg, i, marcos, marco, victima, "frec=%s",
                       fmt_arreglo(buf, sizeof buf, frec, cfg->marcos));
    }
}
```

d) Citas bibliográficas

Fundamentado en Silberschatz et al. (2018).

2.3.8 Algoritmo 7: MFU (Most Frequently Used)

Responsable: Integrante 1 – Angel Ismael Alvarez Catunta

a) Fundamento teórico y principio de operación

MFU expulsa la página con **mayor** contador de usos, bajo el argumento de que una página con contador bajo acaba de llegar y aún se usará (Silberschatz et al., 2018). Usa el mismo contador que LFU y desempate FIFO.

Tabla 15

Ventajas, desventajas, complejidad y anomalía de Belády de MFU

Ventajas	Desventajas
• Simple de implementar.	• Expulsa justamente las páginas más útiles en programas con localidad.
• Útil solo en patrones donde lo más usado ya no se necesita.	• Puede presentar la anomalía de Belády.
Complejidad: O(m) por reemplazo; espacio: un contador por marco.	**Anomalía de Belády: Sí.** Con la cadena de Belády: 9 fallos con 3 marcos y 10 con 4.

b) Modelado y traza completa paso a paso

La Tabla 16 muestra la ejecución de MFU sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 16

Traza de MFU con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`frec=[1,0,0]`
2	0	W	7	**0**	-	**Fallo**	-	`frec=[1,1,0]`
3	1	R	7	0	**1**	**Fallo**	-	`frec=[1,1,1]`
4	2	R	**2**	0	1	**Fallo**	7	`frec=[1,1,1]`
5	0	W	2	0	1	Acierto	-	`frec=[1,2,1]`
6	3	R	2	**3**	1	**Fallo**	0	`frec=[1,1,1]`
7	0	R	2	3	**0**	**Fallo**	1	`frec=[1,1,1]`
8	4	W	**4**	3	0	**Fallo**	2	`frec=[1,1,1]`
9	2	W	4	**2**	0	**Fallo**	3	`frec=[1,1,1]`
10	3	R	4	2	**3**	**Fallo**	0	`frec=[1,1,1]`
11	0	W	**0**	2	3	**Fallo**	4	`frec=[1,1,1]`
12	3	R	0	2	3	Acierto	-	`frec=[1,1,2]`
13	2	R	0	2	3	Acierto	-	`frec=[1,2,2]`
14	1	R	0	**1**	3	**Fallo**	2	`frec=[1,1,2]`
15	2	W	0	1	**2**	**Fallo**	3	`frec=[1,1,1]`
16	0	R	0	1	2	Acierto	-	`frec=[2,1,1]`
17	1	R	0	1	2	Acierto	-	`frec=[2,2,1]`
18	7	R	**7**	1	2	**Fallo**	0	`frec=[1,2,1]`
19	0	W	7	**0**	2	**Fallo**	1	`frec=[1,1,1]`
20	1	R	7	0	**1**	**Fallo**	2	`frec=[1,1,1]`
21	3	W	**3**	0	1	**Fallo**	7	`frec=[1,1,1]`
22	5	R	3	**5**	1	**Fallo**	0	`frec=[1,1,1]`
23	2	W	3	5	**2**	**Fallo**	1	`frec=[1,1,1]`
24	0	W	**0**	5	2	**Fallo**	3	`frec=[1,1,1]`
25	1	R	0	**1**	2	**Fallo**	5	`frec=[1,1,1]`
26	4	R	0	1	**4**	**Fallo**	2	`frec=[1,1,1]`
27	2	R	**2**	1	4	**Fallo**	0	`frec=[1,1,1]`
28	3	W	2	**3**	4	**Fallo**	1	`frec=[1,1,1]`
29	0	W	2	3	**0**	**Fallo**	4	`frec=[1,1,1]`
30	1	R	**1**	3	0	**Fallo**	2	`frec=[1,1,1]`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/07_mfu.txt`.

Resumen estadístico: 25 fallos de página, 5 aciertos, tasa de fallos 83.33 % (tasa de aciertos 16.67 %) y 9 escrituras a disco.

Obtiene 25 fallos, igual que FIFO. Tras expulsar las páginas más usadas, casi todas quedan con contador 1 y el desempate FIFO decide; por eso su comportamiento coincide con el de FIFO.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/mfu.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 7`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * mfu.c - Algoritmo 7: MFU (Most Frequently Used)
 * Responsable: Integrante 1 (Angel Alvarez)
 *
 * Idea: se expulsa la página con MAYOR contador de usos, bajo el
 * argumento de que una página muy usada "ya cumplió su función" y la de
 * contador bajo acaba de llegar y aún se usará. Mismo contador que LFU
 * (se reinicia a 1 al cargar). Empate: la que llegó primero (FIFO).
 *
 * Variables auxiliares: contador de frecuencia de cada marco.
 */
#include "../algoritmos.h"

void simular_mfu(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int frec[MAX_MARCOS] = {0};
    int llegada[MAX_MARCOS] = {0};
    char buf[AUX_LEN];

    resultado_iniciar(r, "MFU", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1;

        if (idx >= 0) {
            frec[idx]++;
            registrar_paso(r, cfg, i, marcos, -1, -1, "frec=%s",
                           fmt_arreglo(buf, sizeof buf, frec, cfg->marcos));
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = 0;
            for (int k = 1; k < cfg->marcos; k++)
                if (frec[k] > frec[marco] ||
                    (frec[k] == frec[marco] && llegada[k] < llegada[marco]))
                    marco = k;
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        frec[marco] = 1;
        llegada[marco] = i;
        registrar_paso(r, cfg, i, marcos, marco, victima, "frec=%s",
                       fmt_arreglo(buf, sizeof buf, frec, cfg->marcos));
    }
}
```

d) Citas bibliográficas
Fundamentado en Silberschatz et al. (2018).

2.3.9 Algoritmo 8: Reloj Mejorado (Enhanced Second Chance)

Responsable: Integrante 4 – Marcus Geuseppe Meza Zamalloa

a) Fundamento teórico y principio de operación
Extiende el Reloj con el par (R, M). El puntero busca primero una página (0,0) sin modificar los bits; si no la encuentra, busca una (0,1) poniendo R = 0 en cada marco que recorre, y repite hasta encontrar víctima. Así prefiere páginas no usadas **y limpias**, que no requieren escritura (Silberschatz et al., 2018).

Tabla 17
Ventajas, desventajas, complejidad y anomalía de Belády de Reloj Mejorado

Ventajas	Desventajas
• Reduce las escrituras a disco.	• Puede requerir hasta cuatro pasadas del puntero.
• Mantiene el bajo costo del Reloj.	• Puede presentar la anomalía de Belády.
Complejidad: O(m) por reemplazo (hasta 4 vueltas); espacio: 2 bits por marco.	**Anomalía de Belády: Sí.** Con la cadena de Belády: 9 fallos con 3 marcos y 10 con 4.

b) Modelado y traza completa paso a paso
La Tabla 18 muestra la ejecución de Reloj Mejorado sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 18

Traza de Reloj Mejorado con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`R=[1,0,0] M=[0,0,0] ptr=M1`
2	0	W	7	**0**	-	**Fallo**	-	`R=[1,1,0] M=[0,1,0] ptr=M1`
3	1	R	7	0	**1**	**Fallo**	-	`R=[1,1,1] M=[0,1,0] ptr=M1`
4	2	R	**2**	0	1	**Fallo**	7	`R=[1,0,0] M=[0,1,0] ptr=M2 (pasada 3)`
5	0	W	2	0	1	Acierto	-	`R=[1,1,0] M=[0,1,0] ptr=M2`
6	3	R	2	0	**3**	**Fallo**	1	`R=[1,1,1] M=[0,1,0] ptr=M1 (pasada 1)`
7	0	R	2	0	3	Acierto	-	`R=[1,1,1] M=[0,1,0] ptr=M1`
8	4	W	**4**	0	3	**Fallo**	2	`R=[1,0,0] M=[1,1,0] ptr=M2 (pasada 3)`
9	2	W	4	0	**2**	**Fallo**	3	`R=[1,0,1] M=[1,1,1] ptr=M1 (pasada 1)`
10	3	R	4	**3**	2	**Fallo**	0	`R=[0,1,1] M=[1,0,1] ptr=M3 (pasada 2)`
11	0	W	**0**	3	2	**Fallo**	4	`R=[1,1,0] M=[1,0,1] ptr=M2 (pasada 2)`
12	3	R	0	3	2	Acierto	-	`R=[1,1,0] M=[1,0,1] ptr=M2`
13	2	R	0	3	2	Acierto	-	`R=[1,1,1] M=[1,0,1] ptr=M2`
14	1	R	0	**1**	2	**Fallo**	3	`R=[0,1,0] M=[1,0,1] ptr=M3 (pasada 3)`
15	2	W	0	1	2	Acierto	-	`R=[0,1,1] M=[1,0,1] ptr=M3`
16	0	R	0	1	2	Acierto	-	`R=[1,1,1] M=[1,0,1] ptr=M3`
17	1	R	0	1	2	Acierto	-	`R=[1,1,1] M=[1,0,1] ptr=M3`
18	7	R	0	**7**	2	**Fallo**	1	`R=[0,1,0] M=[1,0,1] ptr=M3 (pasada 3)`
19	0	W	0	7	2	Acierto	-	`R=[1,1,0] M=[1,0,1] ptr=M3`
20	1	R	0	7	**1**	**Fallo**	2	`R=[1,1,1] M=[1,0,0] ptr=M1 (pasada 2)`
21	3	W	0	**3**	1	**Fallo**	7	`R=[0,1,0] M=[1,1,0] ptr=M3 (pasada 3)`
22	5	R	0	3	**5**	**Fallo**	1	`R=[0,1,1] M=[1,1,0] ptr=M1 (pasada 1)`
23	2	W	**2**	3	5	**Fallo**	0	`R=[1,1,1] M=[1,1,0] ptr=M2 (pasada 2)`
24	0	W	2	3	**0**	**Fallo**	5	`R=[0,0,1] M=[1,1,1] ptr=M1 (pasada 3)`
25	1	R	**1**	3	0	**Fallo**	2	`R=[1,0,1] M=[0,1,1] ptr=M2 (pasada 2)`
26	4	R	1	**4**	0	**Fallo**	3	`R=[1,1,1] M=[0,0,1] ptr=M3 (pasada 2)`
27	2	R	**2**	4	0	**Fallo**	1	`R=[1,0,0] M=[0,0,1] ptr=M2 (pasada 3)`
28	3	W	2	**3**	0	**Fallo**	4	`R=[1,1,0] M=[0,1,1] ptr=M3 (pasada 1)`
29	0	W	2	3	0	Acierto	-	`R=[1,1,1] M=[0,1,1] ptr=M3`
30	1	R	**1**	3	0	**Fallo**	2	`R=[1,0,0] M=[0,1,1] ptr=M2 (pasada 3)`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/08_reloj_mejorado.txt`.

Resumen estadístico: 21 fallos de página, 9 aciertos, tasa de fallos 70.00 % (tasa de aciertos 30.00 %) y 6 escrituras a disco.

Obtiene 21 fallos, como NRU, pero solo 6 escrituras (frente a 7). En el paso 18 las pasadas 1 y 2 no encuentran víctima y en la pasada 3 expulsa la página 1, limpia, protegiendo a las páginas sucias 0 y 2.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/reloj_mejorado.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 8`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * reloj_mejorado.c - Algoritmo 8: Reloj Mejorado (Enhanced Second Chance)
```

```
 * Responsable: Integrante 4 (Marcus Meza)
 *
 * Idea (Silberschatz): Reloj que considera el par (R, M):
 *     (0,0) no usada, limpia  -> mejor víctima
 *     (0,1) no usada, sucia   -> hay que escribirla antes
 *     (1,0) usada, limpia
 *     (1,1) usada, sucia      -> peor víctima
 * Búsqueda con el puntero, desde su posición actual:
 *   Pasada 1: buscar (0,0) dando una vuelta SIN modificar bits.
 *   Pasada 2: buscar (0,1) poniendo R = 0 en cada marco recorrido.
 *   Si no hubo víctima, se repite (ahora todos tienen R = 0).
 * Tras reemplazar, el puntero queda en el marco siguiente.
 *
 * Variables auxiliares: bits R y M, puntero y pasada en que se halló la víctima.
 */
#include "../algoritmos.h"

void simular_reloj_mejorado(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int bit_m[MAX_MARCOS] = {0};
    int puntero = 0;
    char br[AUX_LEN / 3], bm[AUX_LEN / 3];

    resultado_iniciar(r, "Reloj Mejorado", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int w = es_escritura(cfg, i);
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1, pasada = 0;

        if (idx >= 0) {
            bit_r[idx] = 1;
            bit_m[idx] |= w;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s M=%s ptr=M%d",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos), puntero + 1);
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = -1;
            while (marco < 0) {
                /* Pasada 1: (0,0) sin tocar bits */
                pasada++;
                for (int k = 0; k < cfg->marcos; k++) {
                    int c = (puntero + k) % cfg->marcos;
                    if (bit_r[c] == 0 && bit_m[c] == 0) {
                        marco = c;
                        break;
                    }
                }
                if (marco >= 0)
                    break;
                /* Pasada 2: (0,1) limpiando R al pasar */
                pasada++;
                for (int k = 0; k < cfg->marcos; k++) {
                    int c = (puntero + k) % cfg->marcos;
                    if (bit_r[c] == 0 && bit_m[c] == 1) {
                        marco = c;
                        break;
                    }
                    bit_r[c] = 0;
                }
            }
            victima = marcos[marco];
            puntero = (marco + 1) % cfg->marcos;
        }
        marcos[marco] = pagina;
```

```
        bit_r[marco] = 1;
        bit_m[marco] = w;

        if (victima >= 0)
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s ptr=M%d (pasada %d)",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos), puntero + 1, pasada);
        else
            registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s ptr=M%d",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos), puntero + 1);
    }
}
```

d) Citas bibliográficas

Fundamentado en Silberschatz et al. (2018).

2.3.10 Algoritmo 9: Envejecimiento (Aging)

Responsable: Integrante 4 – Marcus Geuseppe Meza Zamalloa

a) Fundamento teórico y principio de operación

Aproxima LRU en software. Cada marco tiene un registro de 8 bits; al final de cada tic el registro se desplaza una posición a la derecha y el bit R entra por la izquierda. El registro de menor valor corresponde a la página usada hace más tiempo y es la víctima (Tanenbaum & Bos, 2015).

Tabla 19

Ventajas, desventajas, complejidad y anomalía de Belády de Envejecimiento

Ventajas	Desventajas
• Muy buena aproximación de LRU con pocos bits.	• Tiene memoria limitada: solo recuerda los últimos 8 tics.
• Bajo costo de hardware (solo el bit R).	• Requiere desplazar todos los registros en cada tic.
Complejidad: O(m) por tic (desplazamiento) y O(m) por reemplazo; espacio: 8 bits por marco.	**Anomalía de Belády:** Posible, pues no es un algoritmo de pila; no se observó en las pruebas realizadas.

b) Modelado y traza completa paso a paso

La Tabla 20 muestra la ejecución de Envejecimiento sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 20

Traza de Envejecimiento con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	`reg=[10000000,-,-]`
2	0	W	7	**0**	-	**Fallo**	-	`reg=[01000000,10000000,-]`
3	1	R	7	0	**1**	**Fallo**	-	`reg=[00100000,01000000,10000000]`
4	2	R	**2**	0	1	**Fallo**	7	`reg=[10000000,00100000,01000000]`
5	0	W	2	0	1	Acierto	-	`reg=[01000000,10010000,00100000]`
6	3	R	2	0	**3**	**Fallo**	1	`reg=[00100000,01001000,10000000]`
7	0	R	2	0	3	Acierto	-	`reg=[00010000,10100100,01000000]`
8	4	W	**4**	0	3	**Fallo**	2	`reg=[10000000,01010010,00100000]`
9	2	W	4	0	**2**	**Fallo**	3	`reg=[01000000,00101001,10000000]`
10	3	R	4	**3**	2	**Fallo**	0	`reg=[00100000,10000000,01000000]`
11	0	W	**0**	3	2	**Fallo**	4	`reg=[10000000,01000000,00100000]`
12	3	R	0	3	2	Acierto	-	`reg=[01000000,10100000,00010000]`
13	2	R	0	3	2	Acierto	-	`reg=[00100000,01010000,10001000]`
14	1	R	**1**	3	2	**Fallo**	0	`reg=[10000000,00101000,01000100]`
15	2	W	1	3	2	Acierto	-	`reg=[01000000,00010100,10100010]`
16	0	R	1	**0**	2	**Fallo**	3	`reg=[00100000,10000000,01010001]`
17	1	R	1	0	2	Acierto	-	`reg=[10010000,01000000,00101000]`
18	7	R	1	0	**7**	**Fallo**	2	`reg=[01001000,00100000,10000000]`
19	0	W	1	0	7	Acierto	-	`reg=[00100100,10010000,01000000]`
20	1	R	1	0	7	Acierto	-	`reg=[10010010,01001000,00100000]`
21	3	W	1	0	**3**	**Fallo**	7	`reg=[01001001,00100100,10000000]`
22	5	R	1	**5**	3	**Fallo**	0	`reg=[00100100,10000000,01000000]`
23	2	W	**2**	5	3	**Fallo**	1	`reg=[10000000,01000000,00100000]`
24	0	W	2	5	**0**	**Fallo**	3	`reg=[01000000,00100000,10000000]`
25	1	R	2	**1**	0	**Fallo**	5	`reg=[00100000,10000000,01000000]`
26	4	R	**4**	1	0	**Fallo**	2	`reg=[10000000,01000000,00100000]`
27	2	R	4	1	**2**	**Fallo**	0	`reg=[01000000,00100000,10000000]`
28	3	W	4	**3**	2	**Fallo**	1	`reg=[00100000,10000000,01000000]`
29	0	W	**0**	3	2	**Fallo**	4	`reg=[10000000,01000000,00100000]`
30	1	R	0	3	**1**	**Fallo**	2	`reg=[01000000,00100000,10000000]`

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/09_aging.txt`.

Resumen estadístico: 22 fallos de página, 8 aciertos, tasa de fallos 73.33 % (tasa de aciertos 26.67 %) y 8 escrituras a disco.

Obtiene 22 fallos y reproduce **exactamente la traza de LRU** en los 30 pasos. La columna *reg* muestra los registros en binario después de cada tic.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/aging.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 9`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * aging.c - Algoritmo 9: Envejecimiento (Aging)
 * Responsable: Integrante 4 (Marcus Meza)
 *
```

```
 * Idea (Tanenbaum 3.4.7): aproximación de LRU en software. Cada marco tiene
 * un registro de AGING_BITS bits. Al final de cada tic de reloj:
 *     reg = (reg >> 1) | (R << (AGING_BITS-1));   R = 0
 * Así el bit más significativo indica "usada en el último tic", el siguiente
 * "usada hace 2 tics", etc. El registro más PEQUEÑO es la página más
 * "vieja": esa es la víctima (empate: menor índice de marco).
 * Página recién cargada: registro 0 y R = 1.
 *
 * Variables auxiliares: registros en binario, DESPUÉS del tic.
 */
#include "../algoritmos.h"

/* Escribe los registros como "[10000000,01000000,-]" */
static const char *fmt_registros(char *buf, size_t len, const unsigned reg[],
                                 const int marcos[], int m)
{
    size_t u = (size_t)snprintf(buf, len, "reg=[");
    for (int k = 0; k < m && u < len; k++) {
        if (marcos[k] == MARCO_VACIO) {
            u += (size_t)snprintf(buf + u, len - u, "-");
        } else {
            for (int b = AGING_BITS - 1; b >= 0 && u < len; b--)
                u += (size_t)snprintf(buf + u, len - u, "%c", (reg[k] >> b) & 1u ? '1' : '0');
        }
        if (u < len)
            u += (size_t)snprintf(buf + u, len - u, "%s", (k + 1 < m) ? "," : "]");
    }
    return buf;
}

void simular_aging(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    unsigned reg[MAX_MARCOS] = {0};
    int bit_r[MAX_MARCOS] = {0};
    char buf[AUX_LEN];
    const unsigned mascara = (1u << AGING_BITS) - 1u;

    resultado_iniciar(r, "Aging", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco = -1, victima = -1;

        if (idx >= 0) {
            bit_r[idx] = 1;
        } else {
            marco = buscar_libre(marcos, cfg->marcos);
            if (marco < 0) {
                marco = 0;
                for (int k = 1; k < cfg->marcos; k++)
                    if (reg[k] < reg[marco])
                        marco = k;
                victima = marcos[marco];
            }
            marcos[marco] = pagina;
            reg[marco] = 0;
            bit_r[marco] = 1;
        }

        /* Tic de reloj: desplazar e insertar R por la izquierda */
        for (int k = 0; k < cfg->marcos; k++) {
            if (marcos[k] == MARCO_VACIO)
                continue;
            reg[k] = ((reg[k] >> 1) | ((unsigned)bit_r[k] << (AGING_BITS - 1))) & mascara;
            bit_r[k] = 0;
        }

        registrar_paso(r, cfg, i, marcos, marco, victima, "%s",
                       fmt_registros(buf, sizeof buf, reg, marcos, cfg->marcos));
    }
```

```
}
```

d) Citas bibliográficas
Fundamentado en Tanenbaum y Bos (2015).

2.3.11 Algoritmo 10: Conjunto de Trabajo (Working Set)

Responsable: Integrante 5 – Elvis Raúl Suca Hancco

a) Fundamento teórico y principio de operación
Basado en el modelo de Denning (1968): una página que no se ha usado en las últimas τ referencias está **fuera del conjunto de trabajo** y es buena candidata a salir. Se implementó la versión de Tanenbaum y Bos (2015): cada marco guarda R y el tiempo del último uso (TUU), los bits R se limpian cada 5 tics y, en un fallo, se recorre la tabla expulsando la primera página con R = 0 y edad mayor que τ = 4.

Tabla 21
Ventajas, desventajas, complejidad y anomalía de Belády de Conjunto de Trabajo

Ventajas	Desventajas
• Relaciona el reemplazo con la localidad real del programa.	• Más costoso: requiere recorrer la tabla y mantener tiempos.
• Base teórica para evitar la hiperpaginación.	• Depende de la elección de τ.
Complejidad: O(m) por reemplazo; espacio: R y un tiempo por marco.	**Anomalía de Belády: Sí, posible** con asignación fija de marcos: en la búsqueda aleatoria se encontraron casos (por ejemplo, 9 → 10 fallos al pasar de 4 a 5 marcos).

b) Modelado y traza completa paso a paso
La Tabla 22 muestra la ejecución de Conjunto de Trabajo sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 22

Traza de Conjunto de Trabajo con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	R=[1,0,0] TUU=[1,-,-] t=1
2	0	W	7	**0**	-	**Fallo**	-	R=[1,1,0] TUU=[1,2,-] t=2
3	1	R	7	0	**1**	**Fallo**	-	R=[1,1,1] TUU=[1,2,3] t=3
4	2	R	**2**	0	1	**Fallo**	7	R=[1,1,1] TUU=[4,4,4] t=4 todas R=1: M1
5	0	W	2	0	1	Acierto	-	R=[1,1,1] TUU=[4,4,4] t=5
6	3	R	**3**	0	1	**Fallo**	2	R=[1,0,0] TUU=[6,4,4] t=6 [reloj: R<-0] todas en WS: la mas vieja
7	0	R	3	0	1	Acierto	-	R=[1,1,0] TUU=[6,4,4] t=7
8	4	W	3	0	**4**	**Fallo**	1	R=[1,1,1] TUU=[8,8,8] t=8 todas en WS: la mas vieja
9	2	W	**2**	0	4	**Fallo**	3	R=[1,1,1] TUU=[9,9,9] t=9 todas R=1: M1
10	3	R	**3**	0	4	**Fallo**	2	R=[1,1,1] TUU=[10,10,10] t=10 todas R=1: M1
11	0	W	3	0	4	Acierto	-	R=[0,1,0] TUU=[10,10,10] t=11 [reloj: R<-0]
12	3	R	3	0	4	Acierto	-	R=[1,1,0] TUU=[10,10,10] t=12
13	2	R	3	0	**2**	**Fallo**	4	R=[1,1,1] TUU=[13,13,13] t=13 todas en WS: la mas vieja
14	1	R	**1**	0	2	**Fallo**	3	R=[1,1,1] TUU=[14,14,14] t=14 todas R=1: M1
15	2	W	1	0	2	Acierto	-	R=[1,1,1] TUU=[14,14,14] t=15
16	0	R	1	0	2	Acierto	-	R=[0,1,0] TUU=[14,14,14] t=16 [reloj: R<-0]
17	1	R	1	0	2	Acierto	-	R=[1,1,0] TUU=[14,14,14] t=17
18	7	R	1	0	**7**	**Fallo**	2	R=[1,1,1] TUU=[18,18,18] t=18 todas en WS: la mas vieja
19	0	W	1	0	7	Acierto	-	R=[1,1,1] TUU=[18,18,18] t=19
20	1	R	1	0	7	Acierto	-	R=[1,1,1] TUU=[18,18,18] t=20
21	3	W	**3**	0	7	**Fallo**	1	R=[1,0,0] TUU=[21,18,18] t=21 [reloj: R<-0] todas en WS: la mas vieja
22	5	R	3	**5**	7	**Fallo**	0	R=[1,1,0] TUU=[22,22,18] t=22 todas en WS: la mas vieja
23	2	W	3	5	**2**	**Fallo**	7	R=[1,1,1] TUU=[23,23,23] t=23 fuera del WS
24	0	W	**0**	5	2	**Fallo**	3	R=[1,1,1] TUU=[24,24,24] t=24 todas R=1: M1
25	1	R	**1**	5	2	**Fallo**	0	R=[1,1,1] TUU=[25,25,25] t=25 todas R=1: M1
26	4	R	**4**	5	2	**Fallo**	1	R=[1,0,0] TUU=[26,25,25] t=26 [reloj: R<-0] todas en WS: la mas vieja
27	2	R	4	5	2	Acierto	-	R=[1,0,1] TUU=[26,25,25] t=27
28	3	W	4	**3**	2	**Fallo**	5	R=[1,1,1] TUU=[28,28,28] t=28 todas en WS: la mas vieja
29	0	W	**0**	3	2	**Fallo**	4	R=[1,1,1] TUU=[29,29,29] t=29 todas R=1: M1
30	1	R	**1**	3	2	**Fallo**	0	R=[1,1,1] TUU=[30,30,30] t=30 todas R=1: M1

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/10_working_set.txt`.

Resumen estadístico: 20 fallos de página, 10 aciertos, tasa de fallos 66.67 % (tasa de aciertos 33.33 %) y 7 escrituras a disco.

Obtiene 20 fallos, distinto de LRU (22). En el paso 23 la página 7 tenía TUU = 18 (edad 5 > τ) y salió por estar fuera del conjunto de trabajo.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/working_set.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 10`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * working_set.c - Algoritmo 10: Working Set (Conjunto de Trabajo)
```

```
 * Responsable: Integrante 5 (Elvis Suca)
 *
 * Idea (Denning 1968; versión de reemplazo de Tanenbaum 3.4.8):
 * el conjunto de trabajo son las páginas usadas en los últimos WS_TAU tics.
 * Una página FUERA del conjunto de trabajo es buena víctima.
 *
 * Cada marco guarda R y TUU (tiempo virtual del último uso). El TUU NO se
 * actualiza en cada acierto (eso sería LRU exacto, muy caro): se actualiza
 * al cargar y durante el barrido en un fallo. Los bits R se limpian con
 * una interrupción de reloj cada PERIODO_RELOJ tics.
 *
 * Fallo con memoria llena, barrido de M1 a Mn (t = paso actual, base 1):
 *   - R = 1            -> TUU = t (está en el conjunto de trabajo).
 *   - R = 0, edad > τ  -> fuera del conjunto: la PRIMERA encontrada es la
 *                         víctima (el barrido sigue solo para actualizar TUU).
 *   - R = 0, edad <= τ -> se recuerda la de mayor edad.
 *   Sin víctima: la de mayor edad con R = 0; si todas tienen R = 1, M1.
 *
 * Variables auxiliares: bits R, TUU de cada marco y aviso de reloj.
 */
#include "../algoritmos.h"

void simular_working_set(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int tuu[MAX_MARCOS];
    char br[AUX_LEN / 3], bt[AUX_LEN / 3];

    resultado_iniciar(r, "Working Set", cfg);
    marcos_vaciar(marcos, cfg->marcos);
    marcos_vaciar(tuu, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int t = i + 1;
        int pagina = cfg->refs[i];
        int reset = (i > 0 && i % PERIODO_RELOJ == 0);
        int idx, marco, victima = -1;
        const char *motivo = "";

        if (reset)
            for (int k = 0; k < cfg->marcos; k++)
                bit_r[k] = 0;

        idx = buscar_pagina(marcos, cfg->marcos, pagina);
        if (idx >= 0) {
            bit_r[idx] = 1;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s TUU=%s t=%d%s",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), t,
                           reset ? " [reloj: R<-0]" : "");
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            int fuera = -1, mas_viejo = -1, edad_max = -1;
            for (int k = 0; k < cfg->marcos; k++) {
                if (bit_r[k] == 1) {
                    tuu[k] = t;
                } else {
                    int edad = t - tuu[k];
                    if (edad > WS_TAU && fuera < 0)
                        fuera = k;
                    if (edad > edad_max) {
                        edad_max = edad;
                        mas_viejo = k;
                    }
                }
            }
            if (fuera >= 0) {
                marco = fuera;
                motivo = " fuera del WS";
```

```
            } else if (mas_viejo >= 0) {
                marco = mas_viejo;
                motivo = " todas en WS: la mas vieja";
            } else {
                marco = 0;
                motivo = " todas R=1: M1";
            }
            victima = marcos[marco];
        }
        marcos[marco] = pagina;
        bit_r[marco] = 1;
        tuu[marco] = t;

        registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s TUU=%s t=%d%s%s",
                       fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                       fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), t,
                       reset ? " [reloj: R<-0]" : "", motivo);
    }
}
```

d) Citas bibliográficas

Fundamentado en Denning (1968); Tanenbaum y Bos (2015).

2.3.12 Algoritmo 11: WSClock (Working Set Clock)

Responsable: Integrante 2 – Haider Alex Carpio Hermoza

a) Fundamento teórico y principio de operación

Combina el Reloj con el conjunto de trabajo. El puntero recorre los marcos: si R = 1, lo pone en 0; si R = 0 y la página es más vieja que τ, la expulsa si está limpia o **programa su escritura** si está sucia, y continúa buscando una víctima limpia (Carr & Hennessy, 1981).

Tabla 23

Ventajas, desventajas, complejidad y anomalía de Belády de WSClock

Ventajas	Desventajas
• Eficiente y con buen rendimiento; base de implementaciones reales.	• Más complejo que el Reloj simple.
• Evita esperar escrituras de páginas sucias.	• Depende de τ.
Complejidad: O(m) por reemplazo (una vuelta del puntero); espacio: R, M y un tiempo por marco.	**Anomalía de Belády: Sí.** Con la cadena de Belády: 9 fallos con 3 marcos y 10 con 4.

b) Modelado y traza completa paso a paso

La Tabla 24 muestra la ejecución de WSClock sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 24

Traza de WSClock con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	R=[1,0,0] M=[0,0,0] TUU=[1,-,-] ptr=M1
2	0	W	7	**0**	-	**Fallo**	-	R=[1,1,0] M=[0,1,0] TUU=[1,2,-] ptr=M1
3	1	R	7	0	**1**	**Fallo**	-	R=[1,1,1] M=[0,1,0] TUU=[1,2,3] ptr=M1
4	2	R	**2**	0	1	**Fallo**	7	R=[1,0,0] M=[0,1,0] TUU=[4,4,4] ptr=M2
5	0	W	2	0	1	Acierto	-	R=[1,1,0] M=[0,1,0] TUU=[4,5,4] ptr=M2
6	3	R	2	0	**3**	**Fallo**	1	R=[0,0,1] M=[0,1,0] TUU=[6,6,6] ptr=M1
7	0	R	2	0	3	Acierto	-	R=[0,1,1] M=[0,1,0] TUU=[6,7,6] ptr=M1
8	4	W	**4**	0	3	**Fallo**	2	R=[1,0,0] M=[1,1,0] TUU=[8,8,8] ptr=M2
9	2	W	4	0	**2**	**Fallo**	3	R=[0,0,1] M=[1,1,1] TUU=[9,8,9] ptr=M1
10	3	R	**3**	0	2	**Fallo**	4	R=[1,0,0] M=[0,1,1] TUU=[10,8,10] ptr=M2
11	0	W	3	0	2	Acierto	-	R=[1,1,0] M=[0,1,1] TUU=[10,11,10] ptr=M2
12	3	R	3	0	2	Acierto	-	R=[1,1,0] M=[0,1,1] TUU=[12,11,10] ptr=M2
13	2	R	3	0	2	Acierto	-	R=[1,1,1] M=[0,1,1] TUU=[12,11,13] ptr=M2
14	1	R	**1**	0	2	**Fallo**	3	R=[1,0,0] M=[0,1,1] TUU=[14,14,14] ptr=M2
15	2	W	1	0	2	Acierto	-	R=[1,0,1] M=[0,1,1] TUU=[14,14,15] ptr=M2
16	0	R	1	0	2	Acierto	-	R=[1,1,1] M=[0,1,1] TUU=[14,16,15] ptr=M2
17	1	R	1	0	2	Acierto	-	R=[1,1,1] M=[0,1,1] TUU=[17,16,15] ptr=M2
18	7	R	**7**	0	2	**Fallo**	1	R=[1,0,0] M=[0,1,1] TUU=[18,18,18] ptr=M2
19	0	W	7	0	2	Acierto	-	R=[1,1,0] M=[0,1,1] TUU=[18,19,18] ptr=M2
20	1	R	**1**	0	2	**Fallo**	7	R=[1,0,0] M=[0,1,1] TUU=[20,20,18] ptr=M2
21	3	W	**3**	0	2	**Fallo**	1	R=[1,0,0] M=[1,1,1] TUU=[21,20,18] ptr=M2
22	5	R	3	**5**	2	**Fallo**	0	R=[0,1,0] M=[1,0,1] TUU=[22,22,18] ptr=M3
23	2	W	3	5	2	Acierto	-	R=[0,1,1] M=[1,0,1] TUU=[22,22,23] ptr=M3
24	0	W	3	**0**	2	**Fallo**	5	R=[0,1,0] M=[1,1,1] TUU=[22,24,24] ptr=M3
25	1	R	3	0	**1**	**Fallo**	2	R=[0,0,1] M=[1,1,0] TUU=[22,25,25] ptr=M1
26	4	R	3	0	**4**	**Fallo**	1	R=[0,0,1] M=[1,1,0] TUU=[22,25,26] ptr=M1
27	2	R	**2**	0	4	**Fallo**	3	R=[1,0,0] M=[0,1,0] TUU=[27,25,27] ptr=M2 escr.prog=1
28	3	W	2	0	**3**	**Fallo**	4	R=[0,0,1] M=[0,1,1] TUU=[28,25,28] ptr=M1
29	0	W	2	0	3	Acierto	-	R=[0,1,1] M=[0,1,1] TUU=[28,29,28] ptr=M1
30	1	R	**1**	0	3	**Fallo**	2	R=[1,0,0] M=[0,1,1] TUU=[30,30,30] ptr=M2

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/11_wsclock.txt`.

Resumen estadístico: 19 fallos de página, 11 aciertos, tasa de fallos 63.33 % (tasa de aciertos 36.67 %) y 4 escrituras a disco.

Obtiene 19 fallos y 4 escrituras, el mejor resultado realizable junto con LFU. Conserva en memoria la página 2, sucia y muy usada, lo que produce un acierto en el paso 23.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/wsclock.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 11`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * wsclock.c - Algoritmo 11: WSClock (Working Set Clock)
 * Responsable: Integrante 2 (Haider Carpio)
 *
```

```
 * Idea (Carr y Hennessy 1981; Tanenbaum 3.4.9): combina Reloj y conjunto de
 * trabajo. Cada marco guarda R, M y TUU (tiempo del último uso).
 * Acierto: R = 1, TUU = t, M |= W.
 * Fallo con memoria llena, el puntero recorre como máximo una vuelta:
 *   - R = 1                    -> R = 0, TUU = t, avanza.
 *   - R = 0, edad > τ, M = 0   -> VÍCTIMA (vieja y limpia).
 *   - R = 0, edad > τ, M = 1   -> se programa su escritura a disco
 *                                 (M = 0, cuenta 1 escritura) y avanza.
 *   - R = 0, edad <= τ         -> está en el conjunto de trabajo, avanza.
 * Si la vuelta termina sin víctima: el primer marco (desde el puntero) con
 * R = 0 y M = 0; si no existe, el marco bajo el puntero.
 *
 * Variables auxiliares: bits R, M, TUU, puntero y escrituras programadas.
 */
#include "../algoritmos.h"

void simular_wsclock(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    int bit_r[MAX_MARCOS] = {0};
    int bit_m[MAX_MARCOS] = {0};
    int tuu[MAX_MARCOS];
    int puntero = 0;
    char br[AUX_LEN / 4], bm[AUX_LEN / 4], bt[AUX_LEN / 4];

    resultado_iniciar(r, "WSClock", cfg);
    marcos_vaciar(marcos, cfg->marcos);
    marcos_vaciar(tuu, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int t = i + 1;
        int pagina = cfg->refs[i];
        int w = es_escritura(cfg, i);
        int idx = buscar_pagina(marcos, cfg->marcos, pagina);
        int marco, victima = -1, programadas = 0;

        if (idx >= 0) {
            bit_r[idx] = 1;
            bit_m[idx] |= w;
            tuu[idx] = t;
            registrar_paso(r, cfg, i, marcos, -1, -1, "R=%s M=%s TUU=%s ptr=M%d",
                           fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                           fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                           fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), puntero + 1);
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            marco = -1;
            for (int k = 0; k < cfg->marcos && marco < 0; k++) {
                int c = puntero;
                if (bit_r[c] == 1) {
                    bit_r[c] = 0;
                    tuu[c] = t;
                } else if (t - tuu[c] > WSCLOCK_TAU) {
                    if (bit_m[c] == 0) {
                        marco = c;                  /* vieja y limpia */
                        break;
                    }
                    bit_m[c] = 0;                   /* programar escritura */
                    marcar_escrito(r, c);
                    programadas++;
                }
                puntero = (puntero + 1) % cfg->marcos;
            }
            if (marco < 0) {
                for (int k = 0; k < cfg->marcos && marco < 0; k++) {
                    int c = (puntero + k) % cfg->marcos;
                    if (bit_r[c] == 0 && bit_m[c] == 0)
                        marco = c;
                }
                if (marco < 0)
```

```
                    marco = puntero;
                }
                victima = marcos[marco];
                puntero = (marco + 1) % cfg->marcos;
            }
            marcos[marco] = pagina;
            bit_r[marco] = 1;
            bit_m[marco] = w;
            tuu[marco] = t;

            if (programadas > 0)
                registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s TUU=%s ptr=M%d escr.prog=%d",
                               fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                               fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                               fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), puntero + 1, programadas);
            else
                registrar_paso(r, cfg, i, marcos, marco, victima, "R=%s M=%s TUU=%s ptr=M%d",
                               fmt_arreglo(br, sizeof br, bit_r, cfg->marcos),
                               fmt_arreglo(bm, sizeof bm, bit_m, cfg->marcos),
                               fmt_arreglo(bt, sizeof bt, tuu, cfg->marcos), puntero + 1);
        }
    }
```

d) Citas bibliográficas

Fundamentado en Carr y Hennessy (1981); Tanenbaum y Bos (2015).

2.3.13 Algoritmo 12: Reemplazo Aleatorio (Random)

Responsable: Integrante 3 – Jhamil Jhonatan Huahualuque Vargas

a) Fundamento teórico y principio de operación

Elige la víctima al azar entre los marcos ocupados. Sirve como referencia de un algoritmo sin información de uso (Belady, 1966). Para que el resultado sea reproducible se usa un generador congruencial lineal propio, $x = (1103515245 \cdot x + 12345) \bmod 2^{31}$, con semilla 42.

Tabla 25

Ventajas, desventajas, complejidad y anomalía de Belády de Reemplazo Aleatorio

Ventajas	Desventajas
• Costo mínimo, sin estructuras auxiliares.	• Resultados variables según la semilla.
• No tiene casos patológicos sistemáticos.	• Puede expulsar páginas muy utilizadas.
Complejidad: O(1) por reemplazo; espacio: el estado del generador.	**Anomalía de Belády: Sí, posible:** fue el algoritmo con más anomalías en la búsqueda aleatoria (91 casos).

b) Modelado y traza completa paso a paso

La Tabla 26 muestra la ejecución de Reemplazo Aleatorio sobre la cadena oficial con 3 marcos, tal como la imprime el simulador en C.

Tabla 26

Traza de Reemplazo Aleatorio con 3 marcos y 30 referencias

Paso	Ref	Op	M1	M2	M3	Resultado	Víctima	Variables auxiliares
1	7	R	**7**	-	-	**Fallo**	-	x=42
2	0	W	7	**0**	-	**Fallo**	-	x=42
3	1	R	7	0	**1**	**Fallo**	-	x=42
4	2	R	**2**	0	1	**Fallo**	7	x=1250496027 -> x mod 3 = 0 (M1)
5	0	W	2	0	1	Acierto	-	x=1250496027
6	3	R	2	0	**3**	**Fallo**	1	x=1116302264 -> x mod 3 = 2 (M3)
7	0	R	2	0	3	Acierto	-	x=1116302264
8	4	W	2	0	**4**	**Fallo**	3	x=1000676753 -> x mod 3 = 2 (M3)
9	2	W	2	0	4	Acierto	-	x=1000676753
10	3	R	2	**3**	4	**Fallo**	0	x=1668674806 -> x mod 3 = 1 (M2)
11	0	W	2	**0**	4	**Fallo**	3	x=908095735 -> x mod 3 = 1 (M2)
12	3	R	**3**	0	4	**Fallo**	2	x=71666532 -> x mod 3 = 0 (M1)
13	2	R	3	0	**2**	**Fallo**	4	x=896336333 -> x mod 3 = 2 (M3)
14	1	R	**1**	0	2	**Fallo**	3	x=1736731266 -> x mod 3 = 0 (M1)
15	2	W	1	0	2	Acierto	-	x=1736731266
16	0	R	1	0	2	Acierto	-	x=1736731266
17	1	R	1	0	2	Acierto	-	x=1736731266
18	7	R	1	0	**7**	**Fallo**	2	x=1314989459 -> x mod 3 = 2 (M3)
19	0	W	1	0	7	Acierto	-	x=1314989459
20	1	R	1	0	7	Acierto	-	x=1314989459
21	3	W	1	0	**3**	**Fallo**	7	x=1535244752 -> x mod 3 = 2 (M3)
22	5	R	1	0	**5**	**Fallo**	3	x=391441865 -> x mod 3 = 2 (M3)
23	2	W	**2**	0	5	**Fallo**	1	x=1108520142 -> x mod 3 = 0 (M1)
24	0	W	2	0	5	Acierto	-	x=1108520142
25	1	R	2	0	**1**	**Fallo**	5	x=1206814703 -> x mod 3 = 2 (M3)
26	4	R	2	**4**	1	**Fallo**	0	x=534045436 -> x mod 3 = 1 (M2)
27	2	R	2	4	1	Acierto	-	x=534045436
28	3	W	**3**	4	1	**Fallo**	2	x=1974836613 -> x mod 3 = 0 (M1)
29	0	W	3	4	**0**	**Fallo**	1	x=238077914 -> x mod 3 = 2 (M3)
30	1	R	3	4	**1**	**Fallo**	0	x=1413854219 -> x mod 3 = 2 (M3)

Nota. En negrita y sombreada, la página cargada en ese paso. Fuente: `resultados/trazas/12_aleatorio.txt`.

Resumen estadístico: 20 fallos de página, 10 aciertos, tasa de fallos 66.67 % (tasa de aciertos 33.33 %) y 8 escrituras a disco.

Obtiene 20 fallos con la semilla 42. La columna auxiliar muestra el número generado y el marco elegido (x mod 3). Con otra semilla el resultado cambia, por lo que no debe interpretarse como un rendimiento estable.

c) Código fuente en lenguaje C para Linux

Módulo `src/algoritmos/aleatorio.c`. Se compila junto con los módulos comunes del Anexo A mediante `make` (gcc -std=c11 -Wall -Wextra -pedantic -O2) y se ejecuta con `./simulador -a 12`, que imprime la traza anterior y el conteo de fallos.

```
/*
 * aleatorio.c - Algoritmo 12: Aleatorio (Random Replacement, RAND)
```

```
 * Responsable: Integrante 3 (Jhamil Huahualuque)
 *
 * Idea: la víctima se elige al azar entre los marcos ocupados. No usa
 * ninguna información de uso; sirve como referencia de "sin inteligencia".
 *
 * Para que el resultado sea REPRODUCIBLE en cualquier compilador se usa un
 * generador congruencial lineal (LCG) propio en lugar de rand():
 *     x = (1103515245 * x + 12345) mod 2^31,   semilla x0 = RANDOM_SEMILLA
 *     víctima = x mod n_marcos
 * El generador solo avanza cuando hay que reemplazar.
 *
 * Variables auxiliares: valor x generado y marco elegido.
 */
#include <stdint.h>
#include "../algoritmos.h"

static uint32_t lcg_siguiente(uint32_t *x)
{
    *x = (1103515245u * *x + 12345u) & 0x7FFFFFFFu;   /* mod 2^31 */
    return *x;
}

void simular_aleatorio(const Config *cfg, Resultado *r)
{
    int marcos[MAX_MARCOS];
    uint32_t x = RANDOM_SEMILLA;

    resultado_iniciar(r, "Aleatorio (RAND)", cfg);
    marcos_vaciar(marcos, cfg->marcos);

    for (int i = 0; i < cfg->n; i++) {
        int pagina = cfg->refs[i];
        int marco, victima = -1;

        if (buscar_pagina(marcos, cfg->marcos, pagina) >= 0) {
            registrar_paso(r, cfg, i, marcos, -1, -1, "x=%lu", (unsigned long)x);
            continue;
        }

        marco = buscar_libre(marcos, cfg->marcos);
        if (marco < 0) {
            uint32_t v = lcg_siguiente(&x);
            marco = (int)(v % (uint32_t)cfg->marcos);
            victima = marcos[marco];
            marcos[marco] = pagina;
            registrar_paso(r, cfg, i, marcos, marco, victima, "x=%lu -> x mod %d = %d (M%d)",
                           (unsigned long)v, cfg->marcos, marco, marco + 1);
            continue;
        }
        marcos[marco] = pagina;
        registrar_paso(r, cfg, i, marcos, marco, victima, "x=%lu", (unsigned long)x);
    }
}
```

d) Citas bibliográficas

Fundamentado en Belady (1966).

2.4 Evaluación comparativa de resultados

La Tabla 27 consolida los resultados de los 12 algoritmos bajo las mismas condiciones (3 marcos, cadena oficial de 30 referencias).

Tabla 27

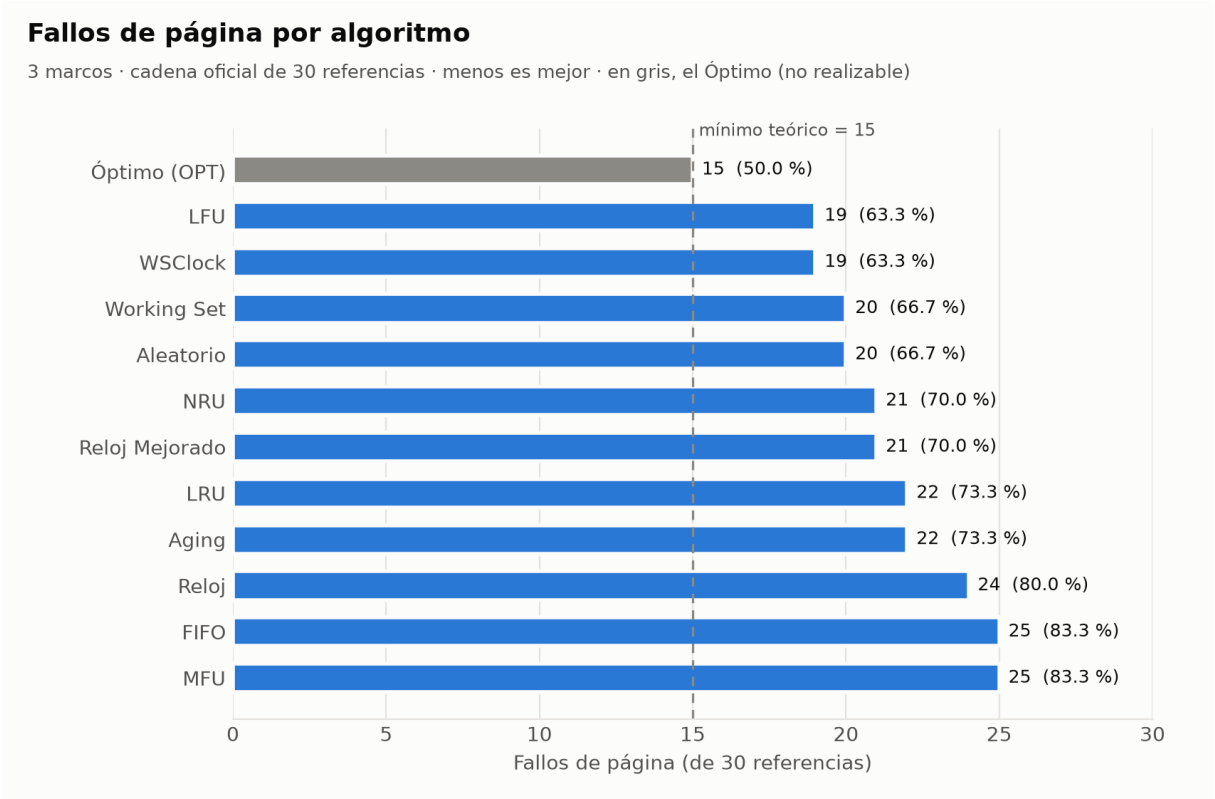
Resultados de los 12 algoritmos con 3 marcos

#	Algoritmo	Responsable	Fallos	Aciertos	% Fallos	Escrituras
1	FIFO	Elvis Suca	25	5	83.33 %	9
2	Óptimo	Jhamil Huahualuque	15	15	50.00 %	6
3	LRU	Jhamil Huahualuque	22	8	73.33 %	8
4	Reloj o Segunda Oportunidad	Haider Carpio	24	6	80.00 %	9
5	NRU	Angel Alvarez	21	9	70.00 %	7
6	LFU	Elvis Suca	19	11	63.33 %	4
7	MFU	Angel Alvarez	25	5	83.33 %	9
8	Reloj Mejorado	Marcus Meza	21	9	70.00 %	6
9	Envejecimiento	Marcus Meza	22	8	73.33 %	8
10	Conjunto de Trabajo	Elvis Suca	20	10	66.67 %	7
11	WSClock	Haider Carpio	19	11	63.33 %	4
12	Reemplazo Aleatorio	Jhamil Huahualuque	20	10	66.67 %	8

Nota. Elaboración propia con el simulador del proyecto.

Figura 2

Fallos de página por algoritmo (3 marcos)

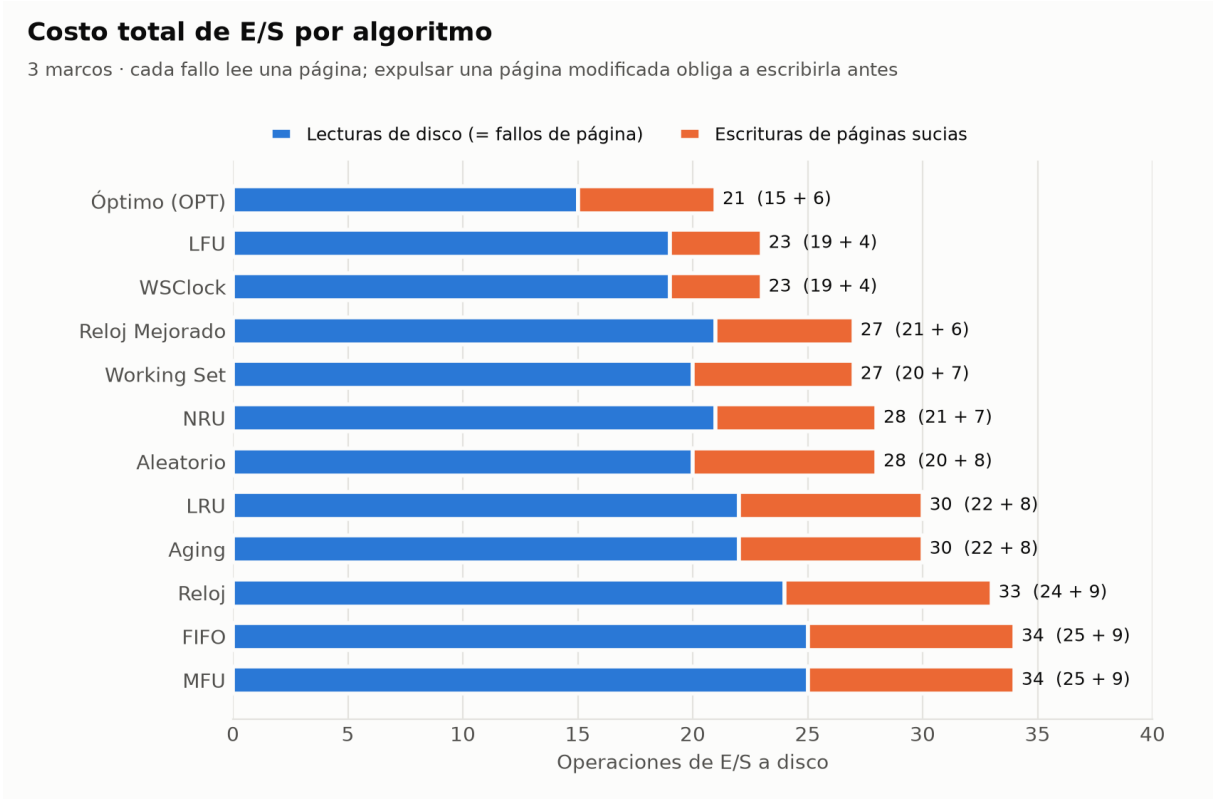


Nota. Elaboración propia con los datos del simulador en C.

El Óptimo marca el mínimo teórico (15 fallos). Los mejores algoritmos realizables son LFU y WSClock (19) y los peores, FIFO y MFU (25): entre el mejor y el peor algoritmo real hay 6 fallos, es decir, el 20 % de las referencias.

Figura 3

Costo total de E/S: lecturas (fallos) más escrituras de páginas sucias

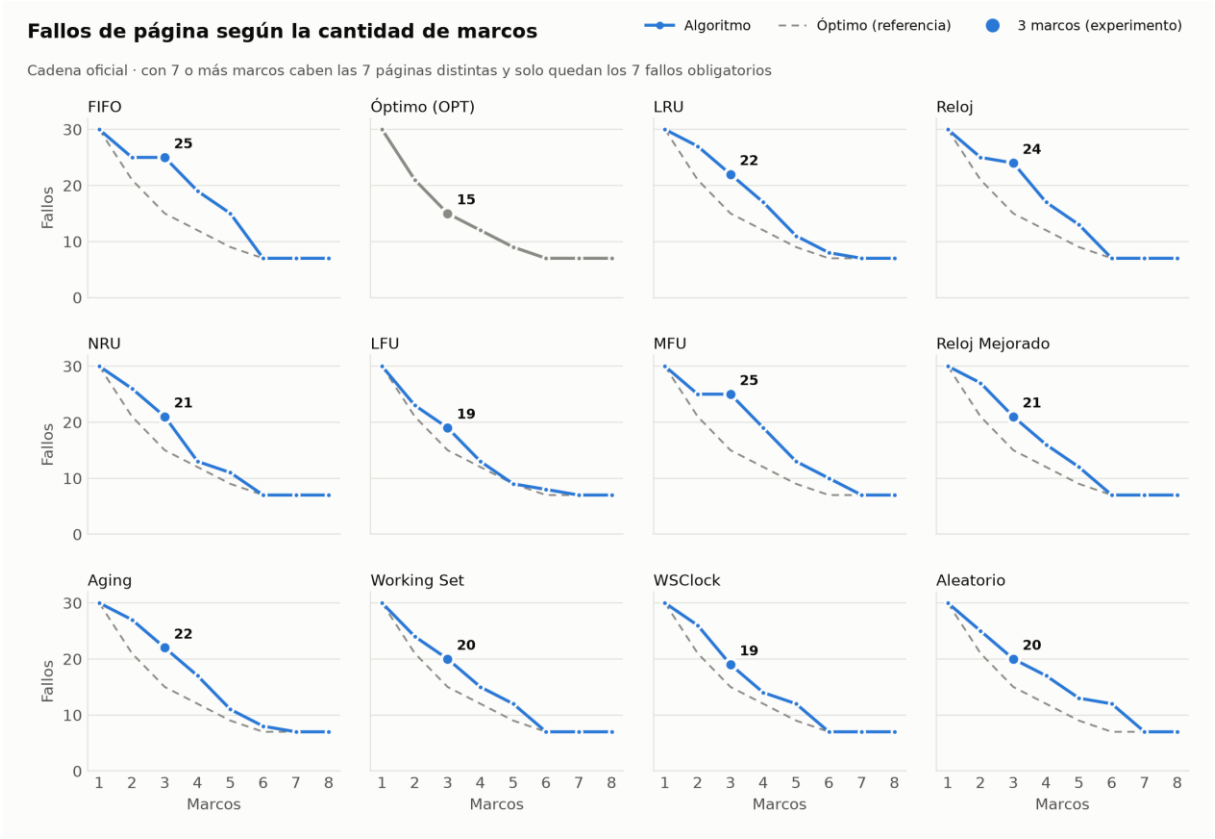


Nota. Elaboración propia con los datos del simulador en C.

Al sumar las escrituras, LFU y WSClock siguen primeros (23 operaciones) y son los que menos escriben (4): LFU conserva la página 0, la más modificada, y WSClock programa la escritura de las páginas sucias antes de expulsarlas. Reloj Mejorado (27) supera a NRU (28) con los mismos fallos, lo que confirma su objetivo de diseño: preferir víctimas limpias.

Figura 4

Fallos de página según la cantidad de marcos (1 a 8)

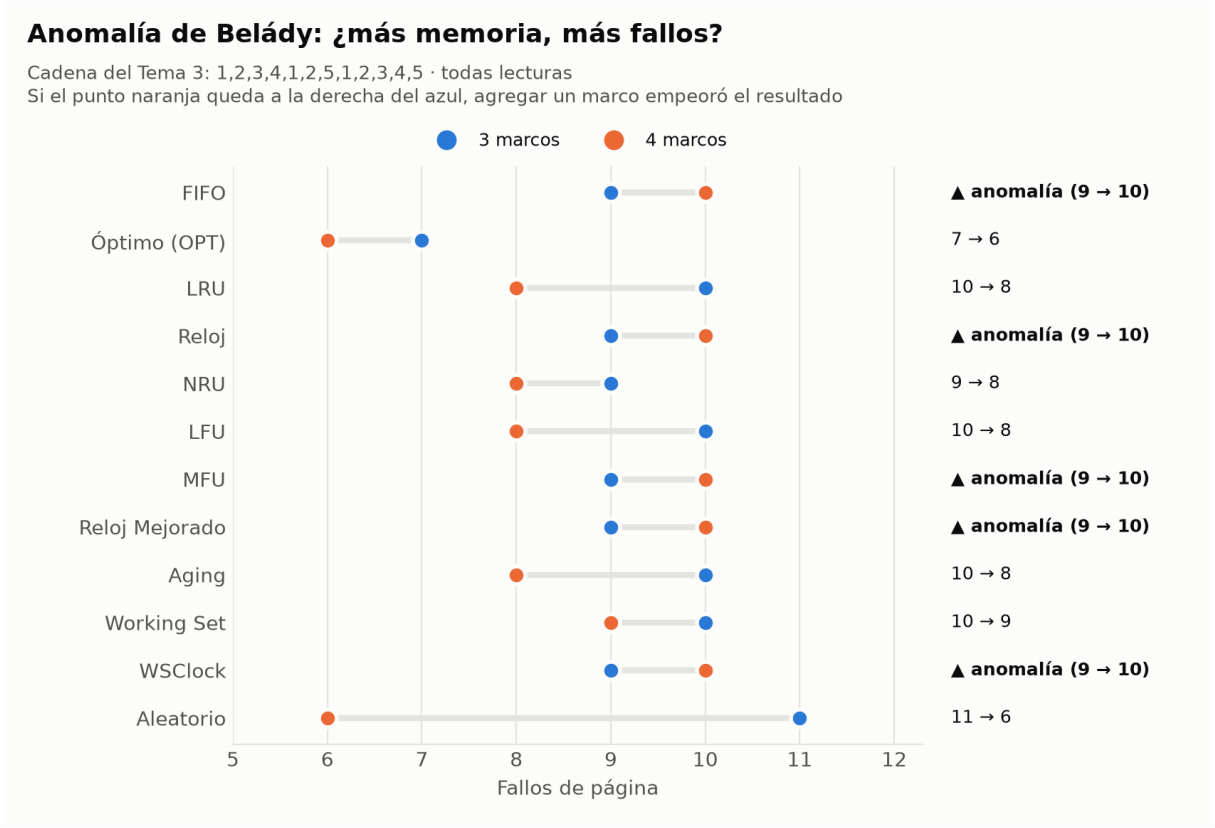


Nota. Elaboración propia con los datos del simulador en C.

Con 1 marco todos obtienen 30 fallos y con 7 o más, 7 fallos (los obligatorios). Con 3 marcos las diferencias son máximas (de 15 a 25), lo que justifica la condición del experimento. FIFO y MFU no mejoran de 2 a 3 marcos, y con 4 marcos NRU y LFU (13) quedan a un fallo del Óptimo (12): el orden de los algoritmos depende de la memoria disponible.

Figura 5

Anomalía de Belády con la cadena del material del curso



Nota. Elaboración propia con los datos del simulador en C.

FIFO, Reloj, MFU, Reloj Mejorado y WSClock empeoran de 9 a 10 fallos al pasar de 3 a 4 marcos. Todos eligen la víctima, en algún momento, por orden de llegada o con un puntero circular. LRU y el Óptimo, por ser algoritmos de pila, nunca presentaron la anomalía, ni en esta cadena ni en 600 cadenas aleatorias.

2.5 Validación del simulador

La exactitud de los resultados se verificó con el script `tests/validar.py` (`make validar`), que compara el simulador con una implementación de referencia escrita por separado en Python a partir de las reglas del experimento. Se superaron **67 de 67 pruebas** (Tabla 28).

Tabla 28

Grupos de pruebas de validación

Grupo	Qué se comprueba	Pruebas
V1	Compilación con gcc sin errores ni advertencias	2
V2	Valores de control (FIFO 25, OPT 15, LRU 22, Reloj 24) y Belády del Tema 3	6
V3	Traza idéntica a la referencia con 1 a 8 marcos (12 algoritmos)	8
V4	Invariantes en cada traza (p. ej., fallos ≥ Óptimo, un fallo cambia un solo marco)	8
V5	Casos de borde (1 marco, sin repeticiones, máximos, etc.)	13
V6	400 cadenas aleatorias con lecturas y escrituras	1
V7	Propiedad de pila (LRU, OPT) y anomalía de Belády	4
V8	Entradas inválidas en la línea de comandos	18
V9	Reproducibilidad y exportación CSV	3
V10	Motor WebAssembly del simulador web idéntico a la referencia (397 casos)	4
	Total	**67/67**

Nota. Salida completa en `docs/evidencias/validacion_paso09.txt` del repositorio.

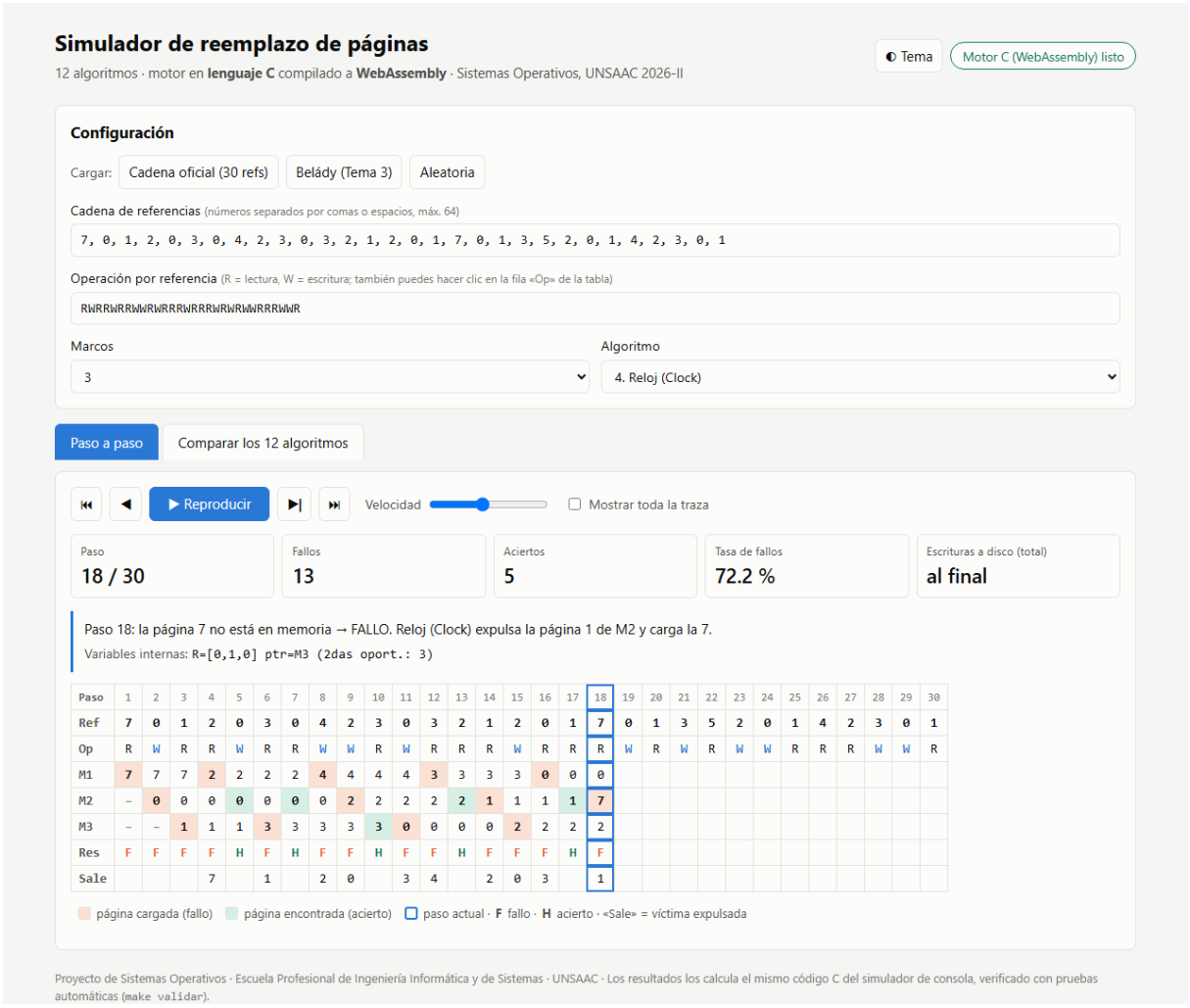
2.6 Simulador web interactivo

Además del programa de consola, el mismo código C se compiló a **WebAssembly** con Emscripten para ofrecer un simulador interactivo en el navegador. La página no reimplementa los algoritmos: llama a las funciones de `src/web/api_web.c`, que ejecutan los 12 módulos en C sin cambios. La prueba V10 comprobó que sus trazas son idénticas a las del programa de consola.

- **Paso a paso:** animación con reproducir/pausa y velocidad, matriz de marcos con la página cargada, el acierto y la víctima, explicación en lenguaje natural y variables internas del algoritmo.
- **Comparación:** gráfico y tabla de los 12 algoritmos con la misma entrada, y prueba de la anomalía de Belády con un marco adicional.
- **Entradas libres:** cadena de hasta 64 referencias, patrón de lecturas/escrituras y de 1 a 8 marcos.
- **Uso:** `make web` compila el motor y `make web-servir` lo abre en http://localhost:8000.

Figura 6

Simulador web: algoritmo del Reloj en el paso 18

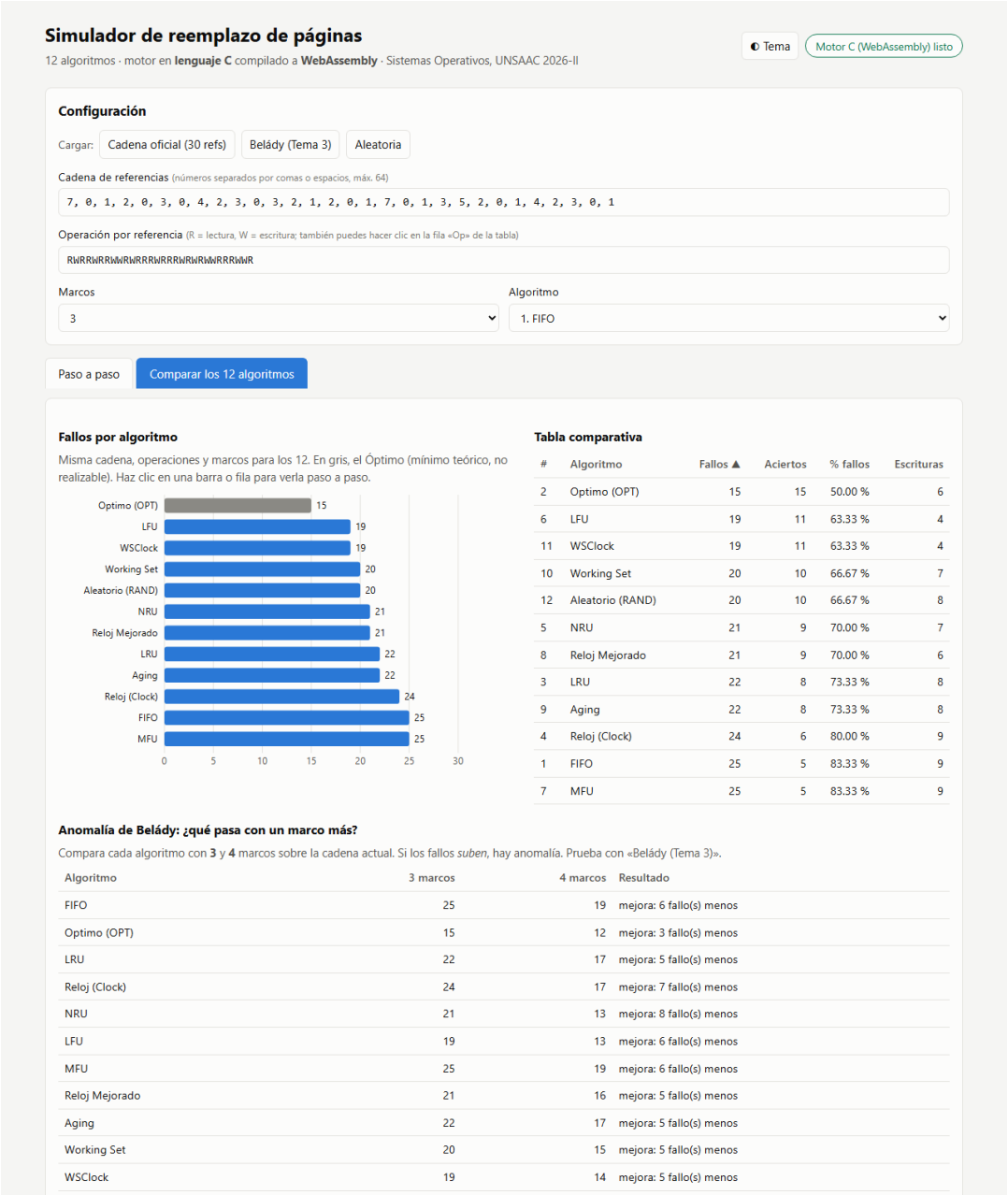


Paso	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Ref	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1	3	5	2	0	1	4	2	3	0	1
Op	R	W	R	R	W	R	R	W	W	R	W	R	R	R	W	R	R	R	W	R	W	R	W	W	R	R	R	W	W	R
M1	7	7	7	2	2	2	2	4	4	4	4	3	3	3	3	0	0	0												
M2	–	0	0	0	0	0	0	0	2	2	2	2	2	1	1	1	1	7												
M3	–	–	1	1	1	3	3	3	3	3	0	0	0	0	2	2	2	2												
Res	F	F	F	F	H	F	H	F	F	H	F	F	H	F	F	F	H	F												
Sale				7		1		2	0		3	4		2	0	3		1												

Nota. Captura del simulador web del proyecto (motor C compilado a WebAssembly).

Figura 7

Simulador web: comparación de los 12 algoritmos y prueba de Belády



Nota. Captura del simulador web del proyecto.

CONCLUSIONES

- Se implementaron en lenguaje C, para Linux, los 12 algoritmos de reemplazo de páginas planteados, en un simulador modular que compila sin advertencias con `gcc -std=c11 -Wall -Wextra -pedantic -O2` y que muestra la traza completa de cada algoritmo con sus variables auxiliares.
- Con la cadena de 30 referencias y 3 marcos, el algoritmo **Óptimo** obtuvo 15 fallos (50 %), el mínimo posible. Entre los algoritmos realizables, **LFU y WSClock** obtuvieron el mejor resultado (19 fallos, 63.33 %) y además el menor número de escrituras a disco (4); **FIFO y MFU** obtuvieron el peor (25 fallos, 83.33 %).
- Contar solo los fallos oculta el costo de las escrituras: **Reloj Mejorado** y **NRU** empataron en 21 fallos, pero Reloj Mejorado realizó una escritura menos al preferir víctimas limpias.
- **Aging** reprodujo exactamente la traza de **LRU** en los 30 pasos, lo que confirma que un registro de 8 bits es una aproximación eficaz de LRU para esta cadena.
- Se verificó experimentalmente la **anomalía de Belády**: con la cadena del material del curso, FIFO, Reloj, MFU, Reloj Mejorado y WSClock pasan de 9 a 10 fallos al aumentar de 3 a 4 marcos, mientras que **LRU y Óptimo**, por ser algoritmos de pila, nunca la presentaron.
- El orden de los algoritmos **depende del número de marcos**: con 4 marcos, NRU y LFU (13 fallos) quedan a un fallo del Óptimo (12) y superan a LRU (17). No existe un algoritmo mejor en todas las condiciones.
- La validación con una implementación de referencia independiente, invariantes, casos de borde y 400 casos aleatorios (67 de 67 pruebas superadas) respalda la exactitud de los resultados.

RECOMENDACIONES

- Repetir el experimento con cadenas obtenidas de programas reales (por ejemplo, con herramientas de trazado en Linux) para evaluar los algoritmos con cargas de trabajo representativas.

- Ampliar el estudio a políticas modernas que combinan recencia y frecuencia, como ARC o LRU-K.
- Modelar varios procesos con asignación global de marcos para estudiar la hiperpaginación de forma directa.
- Ejecutar `make validar` antes de cualquier modificación del simulador para preservar su exactitud.

BIBLIOGRAFÍA

Belady, L. A. (1966). A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal, 5*(2), 78–101. https://doi.org/10.1147/sj.52.0078

Belady, L. A., Nelson, R. A., & Shedler, G. S. (1969). An anomaly in space-time characteristics of certain programs running in a paging machine. *Communications of the ACM, 12*(6), 349–353. https://doi.org/10.1145/363011.363155

Carr, R. W., & Hennessy, J. L. (1981). WSCLOCK—A simple and effective algorithm for virtual memory management. En *Proceedings of the Eighth ACM Symposium on Operating Systems Principles* (pp. 87–95). ACM. https://doi.org/10.1145/800216.806596

Corbató, F. J. (1969). A paging experiment with the Multics system. En H. Feshbach & K. U. Ingard (Eds.), *In honor of Philip M. Morse* (pp. 217–228). MIT Press.

Denning, P. J. (1968). The working set model for program behavior. *Communications of the ACM, 11*(5), 323–333. https://doi.org/10.1145/363095.363141

Denning, P. J. (1970). Virtual memory. *ACM Computing Surveys, 2*(3), 153–189. https://doi.org/10.1145/356571.356573

Kilburn, T., Edwards, D. B. G., Lanigan, M. J., & Sumner, F. H. (1962). One-level storage system. *IRE Transactions on Electronic Computers, EC-11*(2), 223–235. https://doi.org/10.1109/TEC.1962.5219356

Mattson, R. L., Gecsei, J., Slutz, D. R., & Traiger, I. L. (1970). Evaluation techniques for storage hierarchies. *IBM Systems Journal, 9*(2), 78–117. https://doi.org/10.1147/sj.92.0078

Megiddo, N., & Modha, D. S. (2003). ARC: A self-tuning, low overhead replacement cache. En *Proceedings of the 2nd USENIX Conference on File and Storage Technologies* (pp. 115–130). USENIX Association.

Ruiz L., E., & Raffo L., E. (2003). Simulación de la memoria cache. *Industrial Data, 6*(2), 48–55. https://www.redalyc.org/articulo.oa?id=81660207

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4th ed.). Pearson.

ANEXOS

Anexo A. Código fuente común del simulador

Los módulos de cada algoritmo (Capítulo II) se compilan junto con los siguientes archivos. El proyecto completo compila con `make` en Linux sin advertencias.

src/parametros.h

```
/*
 * parametros.h - Parámetros fijos del experimento (PASO 1)
 * Proyecto: Algoritmos de reemplazo de páginas - Sistemas Operativos UNSAAC 2026-II
 *
 * Todos los algoritmos DEBEN usar exactamente estos valores para que la
 * comparación sea válida. Ver docs/PASO_01_parametros.md.
 */
#ifndef PARAMETROS_H
#define PARAMETROS_H

/* ---- Entorno de memoria ---- */
#define NUM_MARCOS      3    /* marcos físicos disponibles              */
#define NUM_REFS        30   /* longitud de la cadena de referencias    */
#define MARCO_VACIO     (-1) /* valor de un marco sin página cargada    */

/* ---- Parámetros específicos de algoritmos ---- */
#define AGING_BITS      8    /* ancho del registro de envejecimiento    */
#define PERIODO_RELOJ   5    /* cada 5 tics se limpian los bits R
                                (NRU y Working Set)                     */
#define WS_TAU          4    /* ventana del conjunto de trabajo (tics)  */
#define WSCLOCK_TAU     4    /* ventana de WSClock (tics)               */
#define RANDOM_SEMILLA  42u  /* semilla del generador LCG propio        */

/* Cadena oficial de 30 referencias (idéntica para los 12 algoritmos) */
extern const int REFS[NUM_REFS];

/* Tipo de acceso de cada referencia: 0 = lectura (R), 1 = escritura (W) */
extern const int ESCRITURA[NUM_REFS];

#endif /* PARAMETROS_H */
```

src/parametros.c

```
/*
 * parametros.c - Definición de la cadena de referencias y patrón de escrituras
 * Ver docs/PASO_01_parametros.md para la justificación de cada valor.
 */
#include "parametros.h"

/*                          paso: 1  2  3  4  5   6  7  8  9 10  11 12 13 14 15  16 17 18 19 20  21 22 23 24 25
26 27 28 29 30 */
const int REFS[NUM_REFS]      = { 7, 0, 1, 2, 0,  3, 0, 4, 2, 3,  0, 3, 2, 1, 2,  0, 1, 7, 0, 1,  3, 5, 2, 0, 1,
4, 2, 3, 0, 1 };
const int ESCRITURA[NUM_REFS] = { 0, 1, 0, 0, 1,  0, 0, 1, 1, 0,  1, 0, 0, 0, 1,  0, 0, 0, 1, 0,  1, 0, 1, 1, 0,
0, 0, 1, 1, 0 };
```

src/comun.h

```
/*
 * comun.h - Tipos y utilidades compartidas por los 12 algoritmos (PASO 3)
 *
 * Cada algoritmo recibe una Config (cadena, patrón de escrituras, nº de marcos)
 * y llena un Resultado con la traza paso a paso. La impresión y las métricas
 * son comunes, así todos los algoritmos se muestran con el mismo formato.
 */
#ifndef COMUN_H
#define COMUN_H

#include <stdio.h>
```

```c
#include "parametros.h"

#define MAX_MARCOS  8    /* permite probar con más marcos (Belády)      */
#define MAX_REFS    64   /* permite cadenas personalizadas más largas   */
#define AUX_LEN     96   /* texto de variables auxiliares por paso      */

/* Datos de entrada de una simulación */
typedef struct {
    const int *refs;       /* cadena de referencias                       */
    const int *escritura;  /* 1 = escritura, 0 = lectura (NULL = lecturas) */
    int n;                 /* cantidad de referencias                     */
    int marcos;            /* cantidad de marcos físicos                  */
} Config;

/* Estado registrado en cada paso de la traza */
typedef struct {
    int pagina;                /* página referenciada                     */
    int es_escritura;          /* tipo de acceso                          */
    int marcos[MAX_MARCOS];    /* contenido de los marcos tras el paso    */
    int marco_cargado;         /* marco donde entró la página, -1 = hit   */
    int victima;               /* página expulsada, -1 si ninguna         */
    char aux[AUX_LEN];         /* variables auxiliares (puntero, bits...) */
} Paso;

/* Resultado completo de un algoritmo */
typedef struct {
    const char *nombre;
    int implementado;          /* 0 = módulo pendiente                    */
    int n;
    int num_marcos;
    int fallos;
    int aciertos;
    int escrituras_disco;      /* páginas sucias escritas a disco         */
    int sucio[MAX_MARCOS];     /* uso interno: marco modificado (dirty)   */
    Paso pasos[MAX_REFS];
} Resultado;

/* ---- Inicialización y registro ---- */
void resultado_iniciar(Resultado *r, const char *nombre, const Config *cfg);
void marcos_vaciar(int marcos[], int m);

/* Registra el paso i. marco_cargado = -1 indica acierto.
 * victima = página expulsada (-1 si se usó un marco libre o fue acierto).
 * aux: texto con formato printf para las variables auxiliares (puede ser NULL). */
void registrar_paso(Resultado *r, const Config *cfg, int i, const int marcos[],
                    int marco_cargado, int victima, const char *aux, ...)
    __attribute__((format(printf, 7, 8)));

/* Escrituras a disco (contabilidad común a todos los algoritmos):
 * registrar_paso() marca un marco como sucio cuando se escribe en él y,
 * al expulsar una página sucia, suma 1 a escrituras_disco.
 * marcar_escrito() se usa cuando un algoritmo (WSClock) escribe la página
 * a disco ANTES de expulsarla: cuenta la escritura y deja el marco limpio. */
void marcar_escrito(Resultado *r, int marco);

/* ---- Búsquedas en los marcos ---- */
int buscar_pagina(const int marcos[], int m, int pagina);  /* índice o -1 */
int buscar_libre(const int marcos[], int m);               /* índice o -1 */
int es_escritura(const Config *cfg, int i);

/* ---- Formato de variables auxiliares ---- */
/* Escribe "[a,b,c]" en buf. Los valores MARCO_VACIO se muestran como '-'. */
const char *fmt_arreglo(char *buf, size_t len, const int v[], int m);

/* ---- Métricas ---- */
double tasa_fallos(const Resultado *r);
double tasa_aciertos(const Resultado *r);

/* ---- Salida ---- */
void imprimir_traza(FILE *f, const Resultado *r, int id, const char *responsable);
void imprimir_resumen(FILE *f, const Resultado *r);

#endif /* COMUN_H */
```

src/comun.c

```
/*
 * comun.c - Implementación de utilidades compartidas (PASO 3)
 */
#include <stdarg.h>
#include <string.h>
#include "comun.h"

void marcos_vaciar(int marcos[], int m)
{
    for (int k = 0; k < m; k++)
        marcos[k] = MARCO_VACIO;
}

void resultado_iniciar(Resultado *r, const char *nombre, const Config *cfg)
{
    memset(r, 0, sizeof *r);
    r->nombre = nombre;
    r->implementado = 1;
    r->n = cfg->n;
    r->num_marcos = cfg->marcos;
}

int es_escritura(const Config *cfg, int i)
{
    return cfg->escritura != NULL && cfg->escritura[i] != 0;
}

void registrar_paso(Resultado *r, const Config *cfg, int i, const int marcos[],
                    int marco_cargado, int victima, const char *aux, ...)
{
    Paso *p = &r->pasos[i];

    p->pagina = cfg->refs[i];
    p->es_escritura = es_escritura(cfg, i);
    for (int k = 0; k < cfg->marcos; k++)
        p->marcos[k] = marcos[k];
    p->marco_cargado = marco_cargado;
    p->victima = victima;

    p->aux[0] = '\0';
    if (aux != NULL) {
        va_list args;
        va_start(args, aux);
        vsnprintf(p->aux, sizeof p->aux, aux, args);
        va_end(args);
    }

    if (marco_cargado < 0) {
        r->aciertos++;
        int idx = buscar_pagina(marcos, cfg->marcos, p->pagina);
        if (idx >= 0 && p->es_escritura)
            r->sucio[idx] = 1;
    } else {
        r->fallos++;
        if (victima >= 0 && r->sucio[marco_cargado])
            r->escrituras_disco++;          /* la víctima sucia se escribe */
        r->sucio[marco_cargado] = p->es_escritura;
    }
}

void marcar_escrito(Resultado *r, int marco)
{
    if (r->sucio[marco]) {
        r->escrituras_disco++;
        r->sucio[marco] = 0;
    }
}

int buscar_pagina(const int marcos[], int m, int pagina)
{
    for (int k = 0; k < m; k++)
        if (marcos[k] == pagina)
```

```
            return k;
    return -1;
}

int buscar_libre(const int marcos[], int m)
{
    return buscar_pagina(marcos, m, MARCO_VACIO);
}

const char *fmt_arreglo(char *buf, size_t len, const int v[], int m)
{
    size_t usado = 0;

    if (len == 0)
        return buf;
    buf[0] = '\0';
    usado += (size_t)snprintf(buf + usado, len - usado, "[");
    for (int k = 0; k < m && usado < len; k++) {
        const char *sep = (k + 1 < m) ? "," : "";
        if (v[k] == MARCO_VACIO)
            usado += (size_t)snprintf(buf + usado, len - usado, "-%s", sep);
        else
            usado += (size_t)snprintf(buf + usado, len - usado, "%d%s", v[k], sep);
    }
    if (usado < len)
        snprintf(buf + usado, len - usado, "]");
    return buf;
}

double tasa_fallos(const Resultado *r)
{
    return r->n > 0 ? 100.0 * r->fallos / r->n : 0.0;
}

double tasa_aciertos(const Resultado *r)
{
    return r->n > 0 ? 100.0 * r->aciertos / r->n : 0.0;
}

static void linea(FILE *f, char c, int ancho)
{
    for (int k = 0; k < ancho; k++)
        fputc(c, f);
    fputc('\n', f);
}

void imprimir_traza(FILE *f, const Resultado *r, int id, const char *responsable)
{
    const int ancho = 100;

    fputc('\n', f);
    linea(f, '=', ancho);
    fprintf(f, " ALGORITMO %d: %s   |   Responsable: %s\n", id, r->nombre, responsable);
    fprintf(f, " Marcos: %d   |   Referencias: %d\n", r->num_marcos, r->n);
    linea(f, '=', ancho);

    if (!r->implementado) {
        fprintf(f, " [PENDIENTE] Este modulo aun no esta implementado (Paso 4).\n");
        return;
    }

    /* Encabezado: Paso | Ref | Op | M1 M2 M3 | Res | Sale | Auxiliares */
    fprintf(f, " Paso | Ref | Op |");
    for (int k = 0; k < r->num_marcos; k++)
        fprintf(f, "  M%d ", k + 1);
    fprintf(f, "| Res | Sale | Variables auxiliares\n");
    linea(f, '-', ancho);

    for (int i = 0; i < r->n; i++) {
        const Paso *p = &r->pasos[i];
        fprintf(f, " %4d | %3d |  %c |", i + 1, p->pagina, p->es_escritura ? 'W' : 'R');
        for (int k = 0; k < r->num_marcos; k++) {
            if (p->marcos[k] == MARCO_VACIO)
```

```
                fprintf(f, "   - ");
            else
                fprintf(f, " %3d%c", p->marcos[k], k == p->marco_cargado ? '*' : ' ');
        }
        fprintf(f, "|  %c  |", p->marco_cargado < 0 ? 'H' : 'F');
        if (p->victima >= 0)
            fprintf(f, " %4d |", p->victima);
        else
            fprintf(f, "    - |");
        fprintf(f, " %s\n", p->aux);
    }
    linea(f, '-', ancho);
    fprintf(f, " (*) pagina cargada en ese paso   F = fallo de pagina   H = acierto\n");
    imprimir_resumen(f, r);
}

void imprimir_resumen(FILE *f, const Resultado *r)
{
    fprintf(f, " Fallos: %d | Aciertos: %d | Tasa de fallos: %.2f %% | Tasa de aciertos: %.2f %%",
            r->fallos, r->aciertos, tasa_fallos(r), tasa_aciertos(r));
    if (r->escrituras_disco > 0)
        fprintf(f, " | Escrituras a disco: %d", r->escrituras_disco);
    fputc('\n', f);
}
```

src/algoritmos.h

```
/*
 * algoritmos.h - Interfaz común de los 12 algoritmos (PASO 3)
 *
 * Todos los algoritmos tienen la MISMA firma:
 *     void simular_xxx(const Config *cfg, Resultado *r);
 * Así main.c puede ejecutarlos desde una tabla (punteros a función).
 */
#ifndef ALGORITMOS_H
#define ALGORITMOS_H

#include "comun.h"

#define NUM_ALGORITMOS 12

typedef void (*FuncSimular)(const Config *cfg, Resultado *r);

typedef struct {
    int id;
    const char *nombre;
    const char *responsable;
    FuncSimular simular;
} Algoritmo;

extern const Algoritmo ALGORITMOS[NUM_ALGORITMOS];

/* Reparto por integrante: ver algoritmos.c (campo responsable) */
void simular_fifo(const Config *cfg, Resultado *r);
void simular_optimo(const Config *cfg, Resultado *r);
void simular_lru(const Config *cfg, Resultado *r);
void simular_reloj(const Config *cfg, Resultado *r);
void simular_nru(const Config *cfg, Resultado *r);
void simular_lfu(const Config *cfg, Resultado *r);
void simular_mfu(const Config *cfg, Resultado *r);
void simular_reloj_mejorado(const Config *cfg, Resultado *r);
void simular_aging(const Config *cfg, Resultado *r);
void simular_working_set(const Config *cfg, Resultado *r);
void simular_wsclock(const Config *cfg, Resultado *r);
void simular_aleatorio(const Config *cfg, Resultado *r);

#endif /* ALGORITMOS_H */
```

src/algoritmos.c

```
/*
 * algoritmos.c - Registro de los 12 algoritmos (PASO 3)
 * Para agregar o reordenar algoritmos solo se modifica esta tabla.
 */
```

```
#include "algoritmos.h"

const Algoritmo ALGORITMOS[NUM_ALGORITMOS] = {
    {  1, "FIFO",              "Elvis Suca", simular_fifo },
    {  2, "Optimo (OPT)",      "Jhamil Huahualuque", simular_optimo },
    {  3, "LRU",               "Jhamil Huahualuque", simular_lru },
    {  4, "Reloj (Clock)",     "Haider Carpio", simular_reloj },
    {  5, "NRU",               "Angel Alvarez", simular_nru },
    {  6, "LFU",               "Elvis Suca", simular_lfu },
    {  7, "MFU",               "Angel Alvarez", simular_mfu },
    {  8, "Reloj Mejorado",    "Marcus Meza", simular_reloj_mejorado },
    {  9, "Aging",             "Marcus Meza", simular_aging },
    { 10, "Working Set",       "Elvis Suca", simular_working_set },
    { 11, "WSClock",           "Haider Carpio", simular_wsclock },
    { 12, "Aleatorio (RAND)",  "Jhamil Huahualuque", simular_aleatorio },
};
```

src/main.c

```
/*
 * main.c - Programa principal del simulador (PASO 3)
 *
 * Uso:
 *   ./simulador                traza de los 12 algoritmos + tabla comparativa
 *   ./simulador -a 1           solo el algoritmo 1 (se puede repetir: -a 1 -a 3)
 *   ./simulador -r             solo la tabla comparativa (sin trazas)
 *   ./simulador -m 4           usar 4 marcos (1..MAX_MARCOS)
 *   ./simulador -c 1,2,3,4,1   cadena personalizada (todas lecturas)
 *   ./simulador -c 1,2,3 -w 0,1,0   patrón de escrituras propio (1 = W)
 *   ./simulador -o salida.csv  exportar resultados a CSV
 *   ./simulador -t trazas.csv  exportar la traza paso a paso a CSV
 *   ./simulador -l             listar algoritmos
 *   ./simulador -h             ayuda
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "algoritmos.h"

static Resultado resultados[NUM_ALGORITMOS];   /* static: evita usar mucha pila */

static void ayuda(const char *prog)
{
    printf("Uso: %s [opciones]\n"
           "  -a N        ejecutar solo el algoritmo N (1..%d), repetible\n"
           "  -r          mostrar solo la tabla comparativa\n"
           "  -m M        numero de marcos (1..%d, por defecto %d)\n"
           "  -c LISTA    cadena personalizada, ej: 1,2,3,4,1,2,5\n"
           "  -w LISTA    patron de escrituras (0=R, 1=W), mismo largo que la cadena\n"
           "  -o ARCHIVO  exportar resultados a CSV\n"
           "  -t ARCHIVO  exportar la traza paso a paso a CSV\n"
           "  -l          listar algoritmos\n"
           "  -h          esta ayuda\n",
           prog, NUM_ALGORITMOS, MAX_MARCOS, NUM_MARCOS);
}

static void listar(void)
{
    printf(" Nro | Algoritmo          | Responsable\n");
    printf("-----+--------------------+--------------------\n");
    for (int k = 0; k < NUM_ALGORITMOS; k++)
        printf(" %3d | %-18s | %s\n", ALGORITMOS[k].id, ALGORITMOS[k].nombre,
               ALGORITMOS[k].responsable);
}

/* Convierte "1,2,3" en un arreglo. Devuelve la cantidad o -1 si es inválida. */
static int parsear_cadena(const char *texto, int destino[], int max)
{
    int n = 0;
    const char *p = texto;

    while (*p != '\0') {
        char *fin;
        long v = strtol(p, &fin, 10);
```

```
        if (fin == p || v < 0 || v > 999 || n >= max)
            return -1;
        destino[n++] = (int)v;
        p = fin;
        if (*p == ',') {
            p++;
            if (*p == '\0')        /* coma final: "1,2," es inválida */
                return -1;
        } else if (*p != '\0') {
            return -1;
        }
    }
    return n > 0 ? n : -1;
}

static int leer_entero(const char *texto, int min, int max, int *valor)
{
    char *fin;
    long v = strtol(texto, &fin, 10);
    if (*texto == '\0' || *fin != '\0' || v < min || v > max)
        return 0;
    *valor = (int)v;
    return 1;
}

static void imprimir_cadena(const Config *cfg)
{
    printf(" Cadena (%d refs): ", cfg->n);
    for (int i = 0; i < cfg->n; i++)
        printf("%d ", cfg->refs[i]);
    printf("\n Operacion:       ");
    for (int i = 0; i < cfg->n; i++)
        printf("%c ", es_escritura(cfg, i) ? 'W' : 'R');
    printf("\n");
}

static void tabla_comparativa(const int seleccion[])
{
    printf("\n");

printf("================================================================================================\n");
    printf("                     TABLA COMPARATIVA DE ALGORITMOS DE REEMPLAZO\n");

printf("================================================================================================\n");
    printf(" Nro | %-18s | %-18s | Fallos | Aciertos | %% Fallos | %% Aciertos | Escr.\n",
           "Algoritmo", "Responsable");
    printf("-----+--------------------+--------------------+--------+----------+----------+------------+------
\n");
    for (int k = 0; k < NUM_ALGORITMOS; k++) {
        const Resultado *r = &resultados[k];
        if (!seleccion[k])
            continue;
        if (!r->implementado) {
            printf(" %3d | %-18s | %-18s | %s\n", ALGORITMOS[k].id, ALGORITMOS[k].nombre,
                   ALGORITMOS[k].responsable, "PENDIENTE (Paso 4)");
            continue;
        }
        printf(" %3d | %-18s | %-18s | %6d | %8d | %7.2f%% | %9.2f%% | %4d\n",
               ALGORITMOS[k].id, ALGORITMOS[k].nombre, ALGORITMOS[k].responsable,
               r->fallos, r->aciertos, tasa_fallos(r), tasa_aciertos(r), r->escrituras_disco);
    }

printf("================================================================================================\n");
}

/* Una fila por paso y algoritmo: sirve para armar las tablas del informe */
static int exportar_trazas_csv(const char *ruta, const int seleccion[], const Config *cfg)
{
    FILE *fp = fopen(ruta, "w");
    if (fp == NULL) {
        perror(ruta);
        return 0;
    }
```

```
    fprintf(fp, "ID,Algoritmo,Paso,Pagina,Operacion");
    for (int k = 0; k < cfg->marcos; k++)
        fprintf(fp, ",M%d", k + 1);
    fprintf(fp, ",Resultado,Victima,Auxiliar\n");

    for (int a = 0; a < NUM_ALGORITMOS; a++) {
        const Resultado *r = &resultados[a];
        if (!seleccion[a] || !r->implementado)
            continue;
        for (int i = 0; i < r->n; i++) {
            const Paso *p = &r->pasos[i];
            fprintf(fp, "%d,\"%s\",%d,%d,%c", ALGORITMOS[a].id, ALGORITMOS[a].nombre,
                    i + 1, p->pagina, p->es_escritura ? 'W' : 'R');
            for (int k = 0; k < cfg->marcos; k++) {
                if (p->marcos[k] == MARCO_VACIO)
                    fprintf(fp, ",-");
                else
                    fprintf(fp, ",%d", p->marcos[k]);
            }
            fprintf(fp, ",%c,", p->marco_cargado < 0 ? 'H' : 'F');
            if (p->victima >= 0)
                fprintf(fp, "%d", p->victima);
            else
                fprintf(fp, "-");
            fprintf(fp, ",\"%s\"\n", p->aux);   /* aux nunca contiene comillas */
        }
    }
    fclose(fp);
    printf(" Trazas exportadas a: %s\n", ruta);
    return 1;
}

static int exportar_csv(const char *ruta, const int seleccion[], const Config *cfg)
{
    FILE *fp = fopen(ruta, "w");
    if (fp == NULL) {
        perror(ruta);
        return 0;
    }
    fprintf(fp, "ID,Algoritmo,Responsable,Marcos,Referencias,Fallos,Aciertos,"
                "Tasa_Fallos_Pct,Tasa_Aciertos_Pct,Escrituras_Disco\n");
    for (int k = 0; k < NUM_ALGORITMOS; k++) {
        const Resultado *r = &resultados[k];
        if (!seleccion[k] || !r->implementado)
            continue;
        fprintf(fp, "%d,\"%s\",\"%s\",%d,%d,%d,%d,%.2f,%.2f,%d\n",
                ALGORITMOS[k].id, ALGORITMOS[k].nombre, ALGORITMOS[k].responsable,
                cfg->marcos, cfg->n, r->fallos, r->aciertos,
                tasa_fallos(r), tasa_aciertos(r), r->escrituras_disco);
    }
    fclose(fp);
    printf(" Resultados exportados a: %s\n", ruta);
    return 1;
}

int main(int argc, char *argv[])
{
    int seleccion[NUM_ALGORITMOS] = {0};
    int alguno_elegido = 0;
    int solo_resumen = 0;
    const char *ruta_csv = NULL;
    const char *ruta_trazas = NULL;
    int cadena_propia[MAX_REFS];
    int escritura_propia[MAX_REFS];
    const char *texto_w = NULL;
    Config cfg = { REFS, ESCRITURA, NUM_REFS, NUM_MARCOS };

    for (int a = 1; a < argc; a++) {
        const char *op = argv[a];
        int necesita_valor = (strcmp(op, "-a") == 0 || strcmp(op, "-m") == 0 ||
                              strcmp(op, "-c") == 0 || strcmp(op, "-o") == 0 ||
                              strcmp(op, "-w") == 0 || strcmp(op, "-t") == 0);
        const char *valor = NULL;
```

```
    if (necesita_valor) {
        if (a + 1 >= argc) {
            fprintf(stderr, "Error: la opcion %s necesita un valor\n", op);
            return 2;
        }
        valor = argv[++a];
    }

    if (strcmp(op, "-h") == 0) {
        ayuda(argv[0]);
        return 0;
    } else if (strcmp(op, "-l") == 0) {
        listar();
        return 0;
    } else if (strcmp(op, "-r") == 0) {
        solo_resumen = 1;
    } else if (strcmp(op, "-a") == 0) {
        int id;
        if (!leer_entero(valor, 1, NUM_ALGORITMOS, &id)) {
            fprintf(stderr, "Error: algoritmo invalido '%s' (1..%d)\n", valor, NUM_ALGORITMOS);
            return 2;
        }
        seleccion[id - 1] = 1;
        alguno_elegido = 1;
    } else if (strcmp(op, "-m") == 0) {
        if (!leer_entero(valor, 1, MAX_MARCOS, &cfg.marcos)) {
            fprintf(stderr, "Error: marcos invalidos '%s' (1..%d)\n", valor, MAX_MARCOS);
            return 2;
        }
    } else if (strcmp(op, "-c") == 0) {
        int n = parsear_cadena(valor, cadena_propia, MAX_REFS);
        if (n < 0) {
            fprintf(stderr, "Error: cadena invalida '%s' (max %d paginas, ej: 1,2,3)\n",
                    valor, MAX_REFS);
            return 2;
        }
        cfg.refs = cadena_propia;
        cfg.escritura = NULL;      /* cadena propia: todas lecturas */
        cfg.n = n;
    } else if (strcmp(op, "-w") == 0) {
        texto_w = valor;           /* se valida al final: depende de -c */
    } else if (strcmp(op, "-t") == 0) {
        ruta_trazas = valor;
    } else if (strcmp(op, "-o") == 0) {
        ruta_csv = valor;
    } else {
        fprintf(stderr, "Opcion desconocida: %s\n", op);
        ayuda(argv[0]);
        return 2;
    }
}

if (texto_w != NULL) {
    int n = parsear_cadena(texto_w, escritura_propia, MAX_REFS);
    int valido = (n == cfg.n);
    for (int i = 0; valido && i < n; i++)
        if (escritura_propia[i] != 0 && escritura_propia[i] != 1)
            valido = 0;
    if (!valido) {
        fprintf(stderr, "Error: patron de escrituras invalido '%s' "
                        "(solo 0/1 y %d valores, igual que la cadena)\n", texto_w, cfg.n);
        return 2;
    }
    cfg.escritura = escritura_propia;
}

if (!alguno_elegido)
    for (int k = 0; k < NUM_ALGORITMOS; k++)
        seleccion[k] = 1;
```

```
printf("*********************************************************************************************\n")
;
    printf("*             SIMULADOR DE ALGORITMOS DE REEMPLAZO DE PAGINAS - Sistemas Operativos UNSAAC
*\n");

printf("*********************************************************************************************\n")
;
    printf(" Marcos: %d\n", cfg.marcos);
    imprimir_cadena(&cfg);

    for (int k = 0; k < NUM_ALGORITMOS; k++) {
        if (!seleccion[k])
            continue;
        ALGORITMOS[k].simular(&cfg, &resultados[k]);
        if (!solo_resumen)
            imprimir_traza(stdout, &resultados[k], ALGORITMOS[k].id, ALGORITMOS[k].responsable);
    }

    tabla_comparativa(seleccion);

    if (ruta_trazas != NULL && !exportar_trazas_csv(ruta_trazas, seleccion, &cfg))
        return 1;
    if (ruta_csv != NULL && !exportar_csv(ruta_csv, seleccion, &cfg))
        return 1;
    return 0;
}
```

src/Makefile

```
# Makefile - Simulador de algoritmos de reemplazo de páginas
# Uso (desde src/):
#   make          compila el simulador y la prueba de parámetros
#   make run      compila y ejecuta el simulador completo
#   make test     compila y ejecuta la prueba de parámetros
#   make validar  validación formal completa (Paso 5)
#   make web      compila el motor C a WebAssembly (Paso 7, requiere Emscripten)
#   make web-servir  abre el simulador web en http://localhost:8000
#   make clean    borra los binarios

SHELL   = /bin/bash
CC      = gcc
CFLAGS  = -std=c11 -Wall -Wextra -pedantic -O2
BUILD   = ../build

# Fuentes
COMUNES = parametros.c comun.c
ALGOS   = $(wildcard algoritmos/*.c)
SIM_SRC = main.c algoritmos.c $(COMUNES) $(ALGOS)
HEADERS = $(wildcard *.h)

# Objetos (se compilan por separado: solo se recompila lo que cambió)
SIM_OBJ = $(patsubst %.c,$(BUILD)/%.o,$(SIM_SRC))

.PHONY: all run test validar validar-rapido resultados web web-servir clean

all: $(BUILD)/simulador $(BUILD)/test_parametros

$(BUILD)/simulador: $(SIM_OBJ)
    $(CC) $(CFLAGS) -o $@ $^

$(BUILD)/test_parametros: $(BUILD)/test_parametros.o $(BUILD)/parametros.o
    $(CC) $(CFLAGS) -o $@ $^

# Regla genérica: .c -> .o (crea la subcarpeta si hace falta)
$(BUILD)/%.o: %.c $(HEADERS)
    @mkdir -p $(dir $@)
    $(CC) $(CFLAGS) -c -o $@ $<

run: $(BUILD)/simulador
    $(BUILD)/simulador

test: $(BUILD)/test_parametros
    $(BUILD)/test_parametros
```

```
# Paso 5: validación formal (requiere python3)
validar:
    python3 ../tests/validar.py

validar-rapido:
    python3 ../tests/validar.py --rapido

# Paso 6: datos del informe (trazas, CSV) en ../resultados/
resultados: $(BUILD)/simulador
    bash ../scripts/generar_resultados.sh

# Paso 7: el MISMO código C compilado a WebAssembly para el simulador web.
# Si emcc no está en el PATH, se carga el entorno de ~/emsdk automáticamente.
WEB_DIR = ../web
WEB_SRC = web/api_web.c algoritmos.c $(COMUNES) $(ALGOS)
EMFLAGS = -sMODULARIZE=1 -sEXPORT_NAME=crearMotor -sENVIRONMENT=web,node \
          -sEXPORTED_RUNTIME_METHODS=ccall,cwrap,UTF8ToString
EMSDK_ENV = $(HOME)/emsdk/emsdk_env.sh

web: $(WEB_DIR)/motor.js

$(WEB_DIR)/motor.js: $(WEB_SRC) $(HEADERS)
    @if ! command -v emcc >/dev/null 2>&1; then \
        if [ -f "$(EMSDK_ENV)" ]; then . "$(EMSDK_ENV)" >/dev/null 2>&1; \
        else echo "Falta Emscripten: ver docs/PASO_07_simulador_web.md"; exit 1; fi; \
    fi; \
    emcc $(CFLAGS) $(EMFLAGS) -o $@ $(WEB_SRC) && echo "Generado: web/motor.js + web/motor.wasm"

web-servir: web
    @echo "Abre http://localhost:8000 en el navegador (Ctrl+C para detener)"
    python3 -m http.server 8000 --directory $(WEB_DIR)

clean:
    rm -rf $(BUILD)
```

src/web/api_web.c

```
/*
 * api_web.c - Interfaz entre la página web y el motor en C (PASO 7)
 *
 * Se compila con Emscripten a WebAssembly junto con los MISMOS módulos que
 * usa el simulador de consola (comun.c, algoritmos.c, los 12 archivos de
 * src/algoritmos/ y parametros.c). La lógica de los 12 algoritmos NO se duplica en JavaScript.
 *
 * Uso desde JavaScript:
 *   1. web_fijar_ref(i, pagina) y web_fijar_escritura(i, 0|1)  para i = 0..n-1
 *   2. web_simular(id, n, marcos)          -> fallos, o -1 si los datos son inválidos
 *   3. web_marco(paso, k), web_marco_cargado(paso), web_victima(paso),
 *      web_aux(paso), web_fallos(), web_escrituras() ...  para leer la traza
 *
 * También compila con gcc (sin Emscripten), lo que permite probarla en Linux.
 */
#include "../algoritmos.h"

#ifdef __EMSCRIPTEN__
#include <emscripten/emscripten.h>
#define EXPORTAR EMSCRIPTEN_KEEPALIVE
#else
#define EXPORTAR
#endif

static int refs[MAX_REFS];
static int escrituras[MAX_REFS];
static Resultado resultado;
static int hay_resultado = 0;

/* Paso válido del último resultado */
static int paso_valido(int paso)
{
    return hay_resultado && paso >= 0 && paso < resultado.n;
}

/* ---------------- Información general ---------------- */
```

```
EXPORTAR int web_num_algoritmos(void) { return NUM_ALGORITMOS; }
EXPORTAR int web_max_marcos(void)     { return MAX_MARCOS; }
EXPORTAR int web_max_refs(void)       { return MAX_REFS; }
EXPORTAR int web_marcos_oficial(void) { return NUM_MARCOS; }
EXPORTAR int web_num_refs_oficial(void) { return NUM_REFS; }

EXPORTAR int web_ref_oficial(int i)
{
    return (i >= 0 && i < NUM_REFS) ? REFS[i] : -1;
}

EXPORTAR int web_escritura_oficial(int i)
{
    return (i >= 0 && i < NUM_REFS) ? ESCRITURA[i] : 0;
}

EXPORTAR const char *web_nombre_algoritmo(int id)
{
    return (id >= 1 && id <= NUM_ALGORITMOS) ? ALGORITMOS[id - 1].nombre : "";
}

EXPORTAR const char *web_responsable(int id)
{
    return (id >= 1 && id <= NUM_ALGORITMOS) ? ALGORITMOS[id - 1].responsable : "";
}

/* ---------------- Entrada ---------------- */

EXPORTAR void web_fijar_ref(int i, int pagina)
{
    if (i >= 0 && i < MAX_REFS)
        refs[i] = pagina;
}

EXPORTAR void web_fijar_escritura(int i, int es_escritura)
{
    if (i >= 0 && i < MAX_REFS)
        escrituras[i] = es_escritura ? 1 : 0;
}

/* Ejecuta el algoritmo 'id' (1..12). Devuelve los fallos o -1 si hay error. */
EXPORTAR int web_simular(int id, int n, int marcos)
{
    Config cfg;

    hay_resultado = 0;
    if (id < 1 || id > NUM_ALGORITMOS || n < 1 || n > MAX_REFS ||
        marcos < 1 || marcos > MAX_MARCOS)
        return -1;
    for (int i = 0; i < n; i++)
        if (refs[i] < 0 || refs[i] > 999)
            return -1;

    cfg.refs = refs;
    cfg.escritura = escrituras;
    cfg.n = n;
    cfg.marcos = marcos;
    ALGORITMOS[id - 1].simular(&cfg, &resultado);
    hay_resultado = resultado.implementado;
    return hay_resultado ? resultado.fallos : -1;
}

/* ---------------- Resultado del último algoritmo simulado ---------------- */

EXPORTAR int web_fallos(void)     { return hay_resultado ? resultado.fallos : -1; }
EXPORTAR int web_aciertos(void)   { return hay_resultado ? resultado.aciertos : -1; }
EXPORTAR int web_escrituras(void) { return hay_resultado ? resultado.escrituras_disco : -1; }
EXPORTAR int web_num_pasos(void)  { return hay_resultado ? resultado.n : 0; }

EXPORTAR int web_pagina(int paso)
{
    return paso_valido(paso) ? resultado.pasos[paso].pagina : -1;
```

```
}

EXPORTAR int web_es_escritura(int paso)
{
    return paso_valido(paso) ? resultado.pasos[paso].es_escritura : 0;
}

/* Contenido del marco k después del paso (MARCO_VACIO = -1) */
EXPORTAR int web_marco(int paso, int k)
{
    if (!paso_valido(paso) || k < 0 || k >= resultado.num_marcos)
        return MARCO_VACIO;
    return resultado.pasos[paso].marcos[k];
}

/* Marco donde se cargó la página (-1 = acierto) */
EXPORTAR int web_marco_cargado(int paso)
{
    return paso_valido(paso) ? resultado.pasos[paso].marco_cargado : -1;
}

/* Página expulsada (-1 = ninguna) */
EXPORTAR int web_victima(int paso)
{
    return paso_valido(paso) ? resultado.pasos[paso].victima : -1;
}

/* Texto de variables auxiliares (puntero, bits R/M, contadores...) */
EXPORTAR const char *web_aux(int paso)
{
    return paso_valido(paso) ? resultado.pasos[paso].aux : "";
}
```

Anexo B. Compilación y ejecución en Linux

```
$ cd src
$ make                       # compila con gcc -std=c11 -Wall -Wextra -pedantic -O2
$ ../build/simulador         # traza de los 12 algoritmos + tabla comparativa
$ ../build/simulador -a 2    # solo el Optimo
$ ../build/simulador -r -m 4 -c 1,2,3,4,1,2,5,1,2,3,4,5   # Belady con 4 marcos
$ ../build/simulador -o resumen.csv -t trazas.csv          # exportar CSV
$ make validar               # 67 pruebas automaticas
$ make web && make web-servir   # simulador web en http://localhost:8000
```

Extracto de la tabla comparativa que imprime el simulador en la terminal de Linux:

```
================================================================================================
                    TABLA COMPARATIVA DE ALGORITMOS DE REEMPLAZO
================================================================================================
 Nro | Algoritmo          | Responsable        | Fallos | Aciertos | % Fallos | % Aciertos | Escr.
-----+--------------------+--------------------+--------+----------+----------+------------+------
   1 | FIFO               | Elvis Suca         |     25 |        5 |   83.33% |     16.67% |    9
   2 | Optimo (OPT)       | Jhamil Huahualuque |     15 |       15 |   50.00% |     50.00% |    6
   3 | LRU                | Jhamil Huahualuque |     22 |        8 |   73.33% |     26.67% |    8
   4 | Reloj (Clock)      | Haider Carpio      |     24 |        6 |   80.00% |     20.00% |    9
   5 | NRU                | Angel Alvarez      |     21 |        9 |   70.00% |     30.00% |    7
   6 | LFU                | Elvis Suca         |     19 |       11 |   63.33% |     36.67% |    4
   7 | MFU                | Angel Alvarez      |     25 |        5 |   83.33% |     16.67% |    9
   8 | Reloj Mejorado     | Marcus Meza        |     21 |        9 |   70.00% |     30.00% |    6
   9 | Aging              | Marcus Meza        |     22 |        8 |   73.33% |     26.67% |    8
  10 | Working Set        | Elvis Suca         |     20 |       10 |   66.67% |     33.33% |    7
  11 | WSClock            | Haider Carpio      |     19 |       11 |   63.33% |     36.67% |    4
  12 | Aleatorio (RAND)   | Jhamil Huahualuque |     20 |       10 |   66.67% |     33.33% |    8
================================================================================================
```